

TypeScript - Les bases

Ajouter des types a JavaScript, simplement.



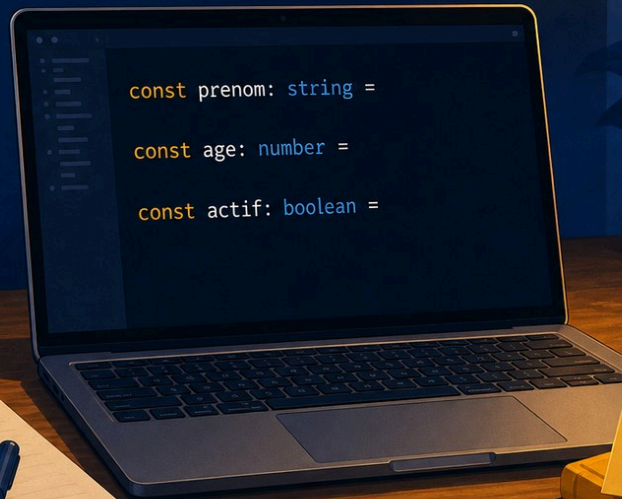
Idees claires



Exemples concrets



Plus de confiance



- Apprendre
- Pratiquer
- Construire

FORMATION TYPESCRIPT - LES BASES

TypeScript

Ajouter des types a JavaScript pour coder plus sur, explique simplement.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - C'est quoi TypeScript ?](#)

- [Ce que ce n'est pas](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Exemple pour sentir](#)

2. [Chapitre 2 - Installer et lancer tsc](#)

- [Ce que ce n'est pas](#)
- [Les fichiers que tu touches](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Le geste quotidien](#)
- [A toi](#)

3. [Chapitre 3 - Les types de base](#)

- [Ce que ce n'est pas](#)
- [Annoter pour sentir](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Choisir le bon type](#)
- [A toi](#)

4. [Chapitre 4 - Annoter une variable \(: type\).](#)

- [Ce que ce n'est pas](#)
- [Syntaxe claire](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Ou annoter en priorite](#)
- [A toi](#)

5. [Chapitre 5 - Les interfaces \(objets types\).](#)

- [Ce que ce n'est pas](#)
- [Proprietes et reemploi](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Faire vivre une interface](#)

- [A toi](#)
6. [Chapitre 6 - Unions et optionnels \(| et ?\)](#)
 - [Ce que ce n'est pas](#)
 - [Petits exemples utiles](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [Honneter le contrat](#)
 - [A toi](#)
 7. [Chapitre 7 - Fonctions typees](#)
 - [Ce que ce n'est pas](#)
 - [Parametres optionnels et defaults](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [Signatures comme documentation](#)
 - [A toi](#)
 8. [Chapitre 8 - Tableaux types](#)
 - [Ce que ce n'est pas](#)
 - [Lire et transformer](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [Listes et coherence](#)
 - [A toi](#)
 9. [Chapitre 9 - Narrowing \(resserrer le type\)](#)
 - [Ce que ce n'est pas](#)
 - [Guards utiles pour debuter](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [Prouver avant d'agir](#)
 - [A toi](#)
 10. [Chapitre 10 - any et unknown](#)
 - [Ce que ce n'est pas](#)
 - [Quand tu touches une donnee floue](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [Sortir du flou sans mentir](#)
 - [A toi](#)
 11. [Chapitre 11 - Lire une erreur du compilateur](#)
 - [Ce que ce n'est pas](#)

- [Methode calme](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Traduire le rouge en action](#)
- [A toi](#)

12. [Chapitre 12 - DOM legerement type](#)

- [Ce que ce n'est pas](#)
- [Gestes du quotidien](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [DOM et types : le duo](#)
- [Lien avec le livre JS](#)
- [A toi](#)

13. [Chapitre 13 - Mini-projet : compteur type](#)

- [Ce que ce n'est pas](#)
- [Squelette](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Construire sans te perdre](#)
- [Checklist avant de montrer](#)
- [A toi](#)

14. [Chapitre 14 - A retenir](#)

- [Ce que ce n'est pas](#)
- [Carte rapide](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Fil rouge DanielCraft](#)
- [Reviser en boucle courte](#)
- [A toi](#)

15. [Chapitre 15 - Atelier : annoter sans paniquer](#)

- [Mission](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai / A toi](#)
- [Deroule detaille](#)

16. [Chapitre 16 - Atelier : une interface pour un vrai objet](#)

- [Mission](#)
- [Ce que ce n'est pas](#)

- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai / A toi](#)
- [Qualite de modele](#)

17. [Chapitre 17 - Atelier : petit projet type de bout en bout](#)

- [Mission](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai / A toi](#)
- [Gerer le temps](#)

18. [Chapitre 18 - Erreurs classiques TypeScript](#)

- [Ce que ce n'est pas](#)
- [Les pieges a coller au mur](#)
- [Petite histoire](#)
- [Erreur classique \(meta\)](#)
- [En vrai](#)
- [Comment utiliser cette liste](#)
- [A toi](#)

19. [Chapitre 19 - Bonnes pratiques debutantes](#)

- [Ce que ce n'est pas](#)
- [Gestes qui marchent](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Ritualiser](#)
- [A toi](#)

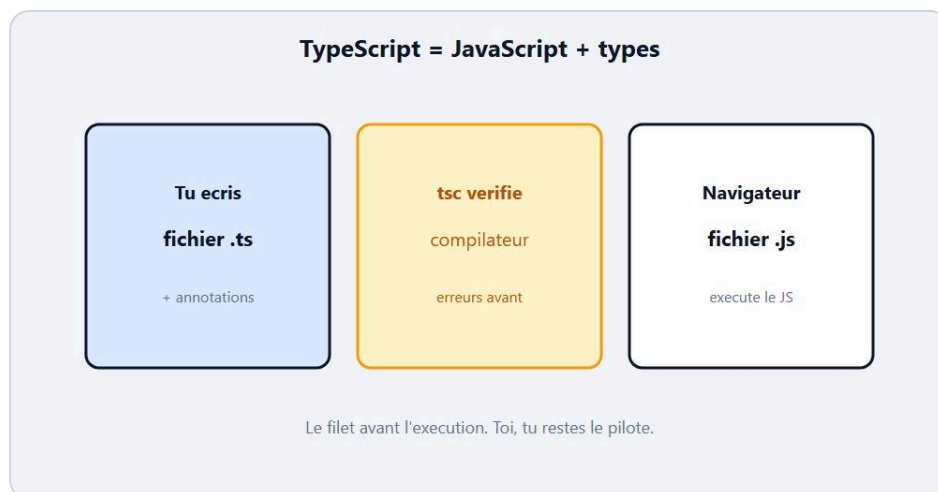
20. [Quiz final - Teste-toi !](#)

- [Questions](#)
- [Reponses](#)
- [Score indicatif](#)

21. [Bravo.](#)

- [Ce que ce n'est pas](#)
- [Ce que tu sais faire maintenant](#)
- [Petite mission](#)
- [Petite histoire](#)
- [La suite quand tu seras pret](#)
- [En vrai](#)
- [A toi](#)

Chapitre 1 - C'est quoi TypeScript ?



TypeScript = JavaScript + types vérifiés avant l'exécution.

Tu connais déjà un peu **JavaScript** : variables, fonctions, tableaux, un peu de DOM. TypeScript, c'est JavaScript avec un filet de sécurité. Tu ajoutes des **types** pour dire clairement ce que chaque valeur est censée être. Avant d'exécuter, un **compilateur** (**tsc**) lit ton code et te signale les incohérences. Ensuite, il produit du JavaScript classique que le navigateur ou Node comprend. Chez DanielCraft, on présente TS comme un garde-fou amical - pas comme un club de theory. Lea l'utilise pour éviter les bugs bêtes avant la demo. Max a compris le jour où **tsc** a refusé un nombre là où il attendait un texte. Sam dit à ses élèves : "TS ne remplace pas JS. Il te force à préciser."

Le geste mental est simple. En JS, tu peux écrire `prenom = 42` par accident et t'en rendre compte trop tard. En TypeScript, tu annonces `prenom: string` et le compilateur proteste si tu ranges un nombre. Tu restes le pilote. TS te rappelle juste les règles que tu as posées. Ce livre suit le livre JavaScript bases : même personnages, même rythme petit-clair-testable. On suppose que tu as déjà touché `const`, `function` et un `if`.

★ A RETENIR

TypeScript = JavaScript + types. Tu écris en `.ts`, tu compiles en `.js`, le navigateur exécute le JS.

Ce que ce n'est pas

Ce n'est pas un autre langage mystérieux sans lien avec JS. Ce n'est pas obligatoire pour "faire du web" dès le jour un. Ce n'est pas un framework (React et cie). Ce n'est pas magie : si tu écris flou, TS te le dira, mais tu dois encore penser. Et ce n'est pas "tout typer parfaitement le premier soir". Commence par annoter des variables et des fonctions simples.

Ce n'est pas non plus Java, malgré le nom proche de "type". Lea rappelle souvent : le vrai travail, c'est encore la logique. Les types aident à ne pas se tromper de forme.

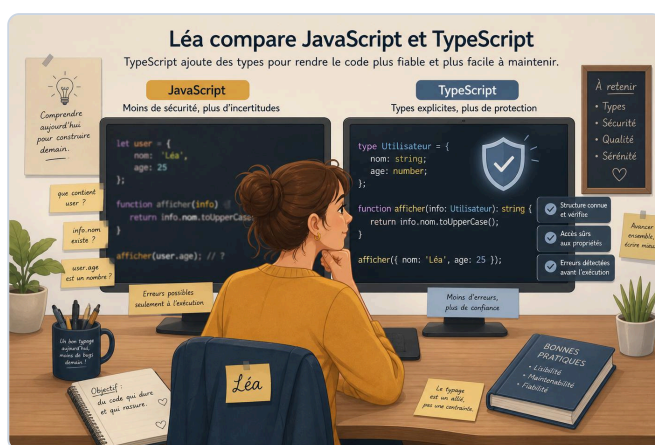
Ce que tu vas savoir faire

A la fin de ce livre, tu sauras installer l'intuition de `tsc` et `tsconfig`, annoter des variables, modeliser des objets avec des **interfaces**, utiliser unions et optionnels, typer fonctions et tableaux, faire un peu de **narrowing**, éviter `any`, lire une erreur du compilateur sans paniquer, toucher le DOM legerement type, et livrer un mini projet compteur ou todo type. Niveau debutant solide. Pas de monorepo. Pas de decorateurs. Juste du TS clair.

Comment lire ce livre

Lis dans l'ordre au debut. Les premiers chapitres posent le sol : idee, install, types, annotations. Le milieu construit interfaces, unions, fonctions, tableaux, narrowing. Les ateliers font faire. Le quiz verifie. A chaque fin, un "A toi". Fais-le. Cinq minutes actives battent une lecture passive.

Chez DanielCraft, on forme des gens qui livrent petit, souvent, proprement - pas des collectionneurs de configs Vite jamais ouvertes. Tu ecris. Tu lances `tsc`. Tu corriges. Ce rythme bat une soiree de videos sans fichier `.ts`.



Exemple : Lea compare JS flou et TS qui garde les types.

Petite histoire

Lea livrait un script de devis. En JS seul, un client passait une chaine la ou le total attendait un nombre. La page affichait `NaN` en plein appel. Avec TypeScript, le meme glissement aurait ete bloque avant la demo. Elle a ajoute trois annotations et un interface `Devis`. Quarante minutes. Le client n'a jamais su qu'il y avait eu un mini drame. Lea, si.

Max voulait juste "moins d'erreurs betes" sur son compteur. Il a renommé `app.js` en `app.ts`, ajoute `number` sur le score, et regarde `tsc` lui dire non quand il ecrivait `score = "dix"`. Sam desactive volontairement le typage en cours : les eleves voient reapparaître le flou. L'idee rentre sans jargon. Personne ne dit "c'est trop". Ils disent "ah, c'est un garde".

Erreur classique

Croire que TypeScript s'exécute directement dans le navigateur comme un fichier `.ts` nu. En vrai, tu compiles (ou un outil le fait pour toi) vers du **JavaScript**. Autre piege : vouloir tout le manuel TypeScript avant d'annoter une seule variable. Ou penser que TS remplace HTML/CSS/JS. Lea garde une regle : une annotation claire bat dix options avancees. DanielCraft insiste : petit, clair, testable.

⚠ ATTENTION

Sans compilation (ou outil equivalent), le navigateur ne "mange" pas le TypeScript. Le produit final reste du JavaScript.

En vrai

Ouvre un petit script JS que tu as deja. Imagine une ligne ou une mauvaise valeur ferait mal : un total, un prenom, un id DOM. Note en une phrase ce que tu voudrais "garantir" comme type. Tu poseras mieux tes priorites pour la suite. Puis dis-toi : ce livre va transformer cette intention en syntaxe : `string`, `number`, `interface`, etc.

A toi

Ecris en trois phrases : (1) un endroit de ton code JS ou un type t'aurait aide, (2) ce que tu acceptes d'apprendre d'abord (annotations, interfaces...), (3) ce que tu ne feras pas encore (generics avances, monorepo). Garde ce papier pour le mini-projet du chapitre 13. Chez DanielCraft, ce petit brief vaut plus qu'une heure de tutorials flous.

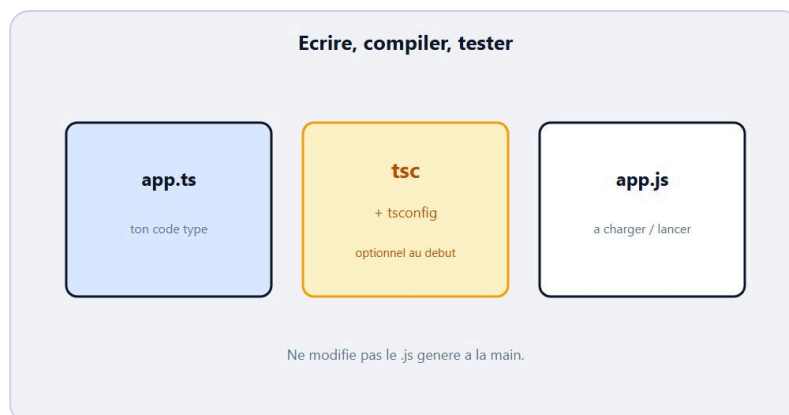
Exemple pour sentir

```
let prenom: string = "Lea";
// prenom = 42; // erreur : number n'est pas assignable a string

function double(n: number): number {
  return n * 2;
}
console.log(double(4));
```

Tu n'as pas besoin de tout comprendre maintenant. L'idee : tu annonces le contrat, le compilateur verifie. Dans ce livre, on demonte ca piece par piece, avec Lea, Max et Sam - et DanielCraft comme fil : petit, clair, testable.

Chapitre 2 - Installer et lancer tsc



Fichier .ts + compilateur : le pont vers du .js.

Pour écrire du TypeScript, tu as besoin d'un outil qui lit les fichiers `.ts` et produit du `.js`. Cet outil s'appelle souvent **tsc** (TypeScript Compiler). Tu peux l'installer via npm (`npm install -g typescript` ou en local dans un projet). Ensuite, tu écris `app.ts`, tu lances `tsc app.ts`, et tu obtiens `app.js`. Chez DanielCraft, on reste volontairement léger : pas de parcours webpack de trois heures. Lea travaille souvent avec un petit dossier et un `tsconfig.json` simple. Max a commencé sans config, juste `tsc compteur.ts`. Sam montre les deux chemins : fichier isolé, puis projet avec config.

Le fichier `tsconfig.json` dit à `tsc` comment se comporter : quel dossier compiler, quel niveau de stricteuse, ou mettre le JS généré. Tu n'as pas besoin de connaître toutes les options. Quelques intuitions suffisent : `strict` te protège plus, `outDir` range le JS ailleurs, `rootDir` dit d'où partent les sources. Si la config manque, `tsc` peut quand même compiler un fichier passe en argument. L'important, c'est le geste : écrire, compiler, lire les erreurs, corriger.

★ A RETENIR

Tu écris en `.ts`. `tsc` vérifie et génère du `.js`. `tsconfig.json` guide le projet sans être magique.

Ce que ce n'est pas

Ce n'est pas obligatoire d'installer vingt extensions VS Code avant d'écrire une ligne. Ce n'est pas "Vite ou rien". Ce n'est pas un cours npm avancé. Ce n'est pas non plus "je copie une config Stack Overflow de 80 lignes". Une config courte et comprise bat une usine opaque. Et ce n'est pas paniquer si le premier `tsc` affiche rouge : c'est normal, c'est le métier.

Les fichiers que tu touches

Un fichier source : `compteur.ts`. Un fichier de sortie : `compteur.js` (généré). Parfois un `tsconfig.json` à la racine. Lea ajoute souvent un script npm `"build": "tsc"` pour ne pas retaper la commande. Max double-clique encore sur sa ligne de commande. Sam insiste : ouvre le `.js` généré une fois pour voir que ce n'est "que" du JavaScript. Le mystère tombe.

```
{
  "compilerOptions": {
    "target": "ES2020",
    "strict": true,
    "outDir": "dist",
    "rootDir": "src"
  },
  "include": ["src/**/*"]
}
```

Tu n'as pas à mémoriser chaque cle. Retiens : `strict` = plus de garde-fous. `outDir` = le JS va dans `dist/`. `include` = quels fichiers regarder.

♦ ASTUCE

Commence par un seul fichier `.ts` et `tsc monfichier.ts`. Ajoute `tsconfig.json` quand le dossier grandit.

Petite histoire

Lea a perdu une heure à chercher pourquoi "rien ne changeait" dans le navigateur. Elle editait le `.ts` mais rechargeait un vieux `.js`. Depuis, elle vérifie la date du fichier genere ou elle lance toujours `tsc` avant le refresh. Max a nommé son fichier `app.TS` en majuscules sur un système tatillon ; Sam lui a montré que la casse compte. DanielCraft répète : le flux écrire -> compiler -> tester est le vrai IDE, pas la couleur de thème.

Erreur classique

Installer TypeScript globalement, oublier de relancer le terminal, puis croire que `tsc` n'existe pas. Ou éditer le `.js` à la main et perdre les changements au prochain compile. Autre piège : coller une config "pro" sans comprendre `strict`, puis abandonner face à une pluie d'erreurs. Lea désactive une option à la fois si besoin, mais elle ne désactive pas le cerveau. Lis le message. Corrige une erreur. Relance.

⚠ ATTENTION

Ne modifie pas le `.js` genere à la main. Tu écris dans le `.ts`, tu recompiles. Sinon tu écrases ton travail.

En vrai

Crée un dossier `hello-ts`. Écris `hello.ts` avec `const msg: string = "salut"; console.log(msg);`. Lance `tsc hello.ts`. Ouvre `hello.js`. Compare. Note une différence (souvent le typage disparaît). Tu as vu le cœur du pipeline.

Le geste quotidien

Lea ouvre son terminal, lance `tsc`, lit la première erreur, corrige, relance. Elle ne cherche pas à "tout configurer parfaitement" avant d'écrire la première ligne. Max a perdu du temps à choisir entre dix templates de starter ; Sam lui a dit de créer un dossier vide et un fichier `.ts`. Chez DanielCraft, le starter parfait est celui que tu comprends. Si tu ne sais pas ce que fait une option de `tsconfig`, ne la copie pas.

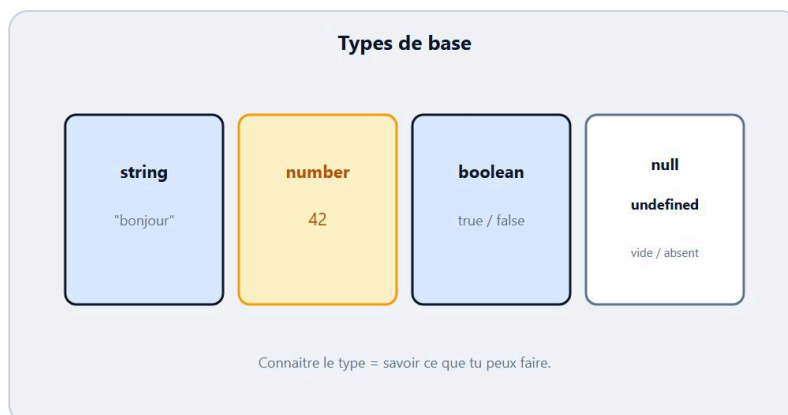
Quand le projet grandit, tu ranges les sources dans `src/` et la sortie dans `dist/`. Tu ajoutes un script npm seulement si tu en as besoin. Tu peux aussi lancer `tsc --watch` pour recompiler a chaque sauvegarde : pratique pendant un atelier. L'important reste le meme : le fichier que tu edites est le `.ts`, le fichier que tu charges dans le navigateur est le `.js` a jour.

Pense aussi a la version de TypeScript. Un message d'erreur un peu different selon les versions, ce n'est pas grave. Le vocabulaire reste proche : type incompatible, propriete manquante, objet possiblement undefined. Apprends a lire ces trois familles et tu seras deja autonome.

A toi

Ecris `age.ts` avec `let age: number = 29;` puis `console.log(age);`. Compile. Change volontairement `age = "vingt";`, recompile, lis l'erreur sans paniquer. Remets le nombre. Chez DanielCraft, ce premier cycle tsc vaut plus qu'un tutorial d'install de trente minutes.

Chapitre 3 - Les types de base



string, number, boolean : les briques de base.

En JavaScript, tu as déjà croisé des **string**, **number**, **boolean**, plus **null** et **undefined**. TypeScript nomme ces formes explicitement pour que le compilateur puisse vérifier. Une string, c'est du texte entre guillemets. Un number, c'est un nombre (entier ou decimal). Un boolean, c'est **true** ou **false**. Chez DanielCraft, on commence toujours par ces trois-là : ils couvrent 80 % des annotations débutantes. Lea type les libelles, Max type les totaux, Sam type les interrupteurs **actif** / **inactif**.

Il existe aussi **null** (vide volontaire) et **undefined** (pas encore défini). Tu les rencontreras surtout avec les unions et les optionnels plus tard. Pour l'instant, retiens : connaître le type, c'est savoir ce que tu peux faire avec la boîte. Tu ne fais pas `.toUpperCase()` sur un nombre. Tu ne multiplies pas deux booleans en espérant un prix. Le type n'est pas une décoration : c'est une promesse sur les opérations autorisées.

★ A RETENIR

string = texte. number = nombre. boolean = vrai/faux. Ces trois types portent déjà beaucoup de code débutant.

Ce que ce n'est pas

Ce n'est pas encore les objets complexes, les tableaux, ni **any**. Ce n'est pas "tous les types du manuel" (tuple, enum, never...). Ce n'est pas non plus inventer des types fantaisie sans besoin. Et ce n'est pas croire que TypeScript change la valeur à l'exécution : les types s'effacent au compile. Le runtime reste du JS. Lea le rappelle aux stagiaires : "après **tsc**, il ne reste que du JavaScript honnête".

Annoter pour sentir

```
let titre: string = "Devis plomberie";
let total: number = 120;
let paye: boolean = false;

// titre = 10; // erreur
// total = "cent"; // erreur
paye = true;
```

Lea écrit parfois sans annotation quand l'inférence suffit : `let ville = "Lyon"` est déjà vu comme `string`. Mais pour apprendre, annoter à la main force le réflexe. Max annote toujours au début. Sam alterne : "d'abord explicite, ensuite tu laisses inférer".

```
let score = 0; // infere : number
score = score + 1;
```

Tu peux aussi croiser les trois types dans une petite fiche :

```
const prenom: string = "Sam";
const sessions: number = 3;
const certifie: boolean = true;
console.log(prenom, sessions, certifie);
```

Petite histoire

Max stockait un prix en `string` `"45"` puis tentait `prix * 1.2`. En JS, ça peut "marcher" bizarrement ou produire des surprises. En TS, si `prix` est `string`, le compilateur te regarde de travers selon le contexte. Il a passé `prix` en `number` et le calcul est devenu honnête. Lea a un tableau mental : texte pour afficher, nombre pour calculer, boolean pour brancher. Sam colle ce tableau au mur de la salle. DanielCraft adore ces cartes simples : elles évitent dix débats abstraits.

Erreur classique

Confondre `number` et `string` numérique `"42"`. Utiliser `Boolean` (objet) au lieu de `boolean` (type primitif) par copie de doc. Ou typer tout en `string` "parce que c'est plus simple" et perdre les calculs. Autre piège : croire que `null` et `undefined` sont la même chose. Proches, pas identiques. On y revient avec `|`.

⚠ ATTENTION

`"42"` est une `string`. `42` est un `number`. Pour calculer, tu veux le second. Pour afficher, le premier peut suffire.

En vrai

Dans la console TS (ou un fichier), déclare trois variables : `prenom`, `age`, `inscrit` avec les bons types. Affiche-les. Tente une mauvaise assignation et lis le message. Note le mot-clé du type dans l'erreur : c'est ton vocabulaire. Puis change volontairement un type et regarde comment le message évolue. Ce petit jeu bat une page de théorie.

Choisir le bon type

Devant une valeur, pose trois questions. Est-ce du texte à afficher ou concaténer ? -> `string`. Est-ce une quantité à calculer ? -> `number`. Est-ce un interrupteur oui/non ? -> `boolean`. Si la réponse est "parfois texte, parfois nombre", tu n'es plus sûr d'un type de base seul : tu vises une union (chapitre suivant sur `|`). Lea applique ce test en revue de code. Max l'écrit encore sur un post-it. Sam le fait dire à voix haute avant d'annoter.

Les nombres en JavaScript / TypeScript ne distinguent pas entier et flottant comme certains langages. `1` et `1.5` sont tous deux `number`. Pour de l'argent, tu resteras prudent (arrondis), mais le type reste `number` au debut. Les booleans ne sont pas des strings `"true"`. Si tu lis un formulaire, tu convertis explicitement. Enfin, souviens-toi : le type guide le compilateur. Il disparaît au runtime. Ton `boolean` compile devient un vrai/faux JS classique. C'est pour ça que TS n'est pas "un autre moteur" : c'est un contrôleur avant le moteur.

A toi

Ecris un mini "fiche contact" : `nom: string`, `telephone: string`, `age: number`, `clientActif: boolean`. Remplis avec tes valeurs. Change un type volontairement, corrige. Chez DanielCraft, cette fiche reviendra sous forme d'interface au chapitre 5. Garde le fichier : tu le reprendras.

Chapitre 4 - Annoter une variable (: type)



: type sur une variable = contrat lisible.

L'**annotation** de type, c'est le `: type` apres le nom. Tu ecris `let score: number = 0`. Tu dis au compilateur : cette boite ne doit contenir que des nombres. Si plus tard tu fais `score = "dix"`, TypeScript refuse. Chez DanielCraft, on traite l'annotation comme une etiquette honnete sur un carton : pas de surprise a l'ouverture. Lea annote les entrees importantes. Max annote tout au debut pour apprendre. Sam montre aussi l'**inference** : parfois TS devine tout seul.

Deux styles cohabitent. Style explicite : tu ecris le type. Style inference : tu initialises clairement et TS deduit. Les deux sont valides. Pour debuter, explicite aide a voir le contrat. Ensuite, tu laisses inferer quand c'est evident. L'annotation brille surtout sur les parametres de fonctions et les valeurs qui changent au fil du script.

★ A RETENIR

: type annonce le contrat de la variable. Le compilateur refuse ce qui casse le contrat.

Ce que ce n'est pas

Ce n'est pas obligatoire sur chaque ligne. Ce n'est pas une decoration "pro". Ce n'est pas non plus `as any` pour faire taire l'erreur. Et ce n'est pas changer le type a l'execution : apres compile, l'annotation disparaît. Le JS restant n'a plus le `: number`.

Syntaxe claire

```
let message: string = "Bonjour";
let compteur: number = 0;
const PI: number = 3.14;
let pret: boolean = false;

compteur = compteur + 1;
// compteur = "un"; // erreur de compilation
```

Avec `const`, la valeur ne change pas, mais le type reste utile pour documenter. Lea prefere `const` des que possible, comme en JS. Max a trop utilise `let` partout ; Sam l'a ramene a `const` + annotation.

```
const ville = "Lyon"; // string infere
let etape: string;
etape = "paiement"; // ok
// etape = 3; // erreur
```

Tu peux declarer sans valeur initiale si tu annotes : TypeScript sait deja la forme attendue. Sans annotation et sans valeur, c'est plus flou - evite.

♦ ASTUCE

Si tu declares sans initialiser, annote. Sinon initialise clairement pour laisser inferer.

Petite histoire

Lea avait une variable `statut` qui passait de `"ok"` a `1` selon les branches. En JS, silence. En TS avec `statut: string`, le `1` a ete refuse. Elle a cree une union plus tard (`"ok" | "ko"`). Mais le premier cran, c'etait l'annotation simple. Max a annote `total: number` et a vu disparaitre trois bugs de concatenation `"10" + 5`. Sam projette avant/apres : la salle comprend mieux qu'avec un discours sur "le systeme de types".

Erreur classique

Annoter `any` partout "pour que ca compile". Reannoter une inference evidente en empilant du bruit. Oublier que `let x;` sans type ni valeur devient facilement problematique en mode strict. Autre piege : confondre annotation (`: string`) et assertion forcee (`as string`). La premiere demande une vraie verification. La seconde dit "fais-moi confiance" - a utiliser avec parcimonie, rarement en debutant.

⚠ ATTENTION

`as string` n'est pas une annotation pedagogique. Prefere `: string` et des valeurs coherentes. Le forage cache les vrais problemes.

En vrai

Prends trois `let` de ton dernier script JS. Ajoute des annotations. Compile. Si une erreur apparait, celebre : tu as trouve un flou. Corrige la valeur ou le type, pas le silence.

Ou annoter en priorite

Annoter partout fatigue et ajoute du bruit. Annoter nulle part laisse le flou. Le bon milieu debutant : parametres de fonctions, variables d'etat qui vivent longtemps (`score`, `todos`), valeurs qui traversent plusieurs fonctions. Lea annote les frontieres. Max annote aussi les locales le temps d'apprendre, puis allège. Sam dit : "si tu hesites sur le type, c'est peut-etre que la variable fait trop de choses".

L'inference n'est pas ton ennemie. `const n = 3` est clairement un number. Tu n'as pas besoin de `const n: number = 3` sauf pour pedagogie. En revanche, `let data;` sans valeur est un piege : precise le type ou initialise tout de suite. En mode strict, TypeScript te pousse dans ce sens. Accepte la pression : elle remplace des bugs plus tard.

Quand une annotation et une valeur se contredisent, crois le compilateur. Soit tu as menti sur le type, soit tu as mis la mauvaise valeur. Corrige l'un des deux, pas le message avec un `as`.

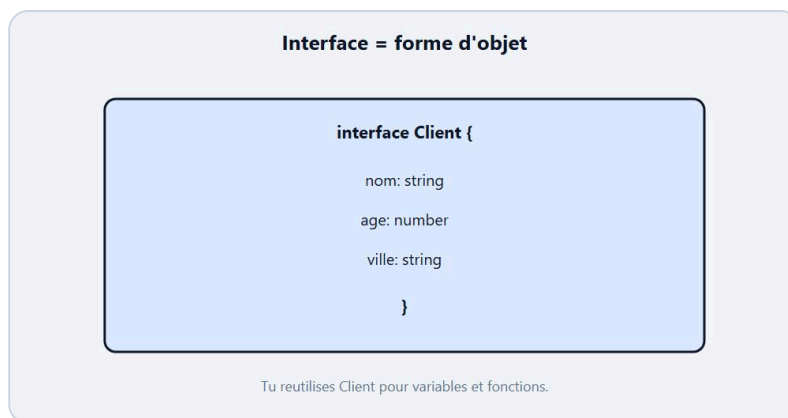
A toi

Ecris :

```
let titre: string;  
let pages: number;  
let publie: boolean;
```

Assigne des valeurs coherentes, puis tente une mauvaise assignation sur chacune. Lis les trois messages. Note le pattern. Chez DanielCraft, lire l'erreur fait partie du typage autant que l'ecrire.

Chapitre 5 - Les interfaces (objets types)



Interface : la forme d'un objet, écrite clairement.

Un objet JS, c'est des étiquettes + des valeurs : `{ nom: "Lea", age: 29 }`. Une **interface** TypeScript décrit la forme attendue de cet objet. Tu dis : un `Client` a un `nom` `string` et un `age` `number`. Ensuite, toute variable `Client` doit respecter ce contrat. Chez DanielCraft, l'interface est la fiche produit du code : claire, partageable, stable. Lea crée `interface Devis` avant d'écrire la logique. Max a longtemps passé des objets "au feeling" ; une interface lui a évité d'oublier `email`. Sam fait dessiner la fiche au tableau avant le clavier.

```
interface Client {
  nom: string;
  age: number;
  ville: string;
}

const c: Client = {
  nom: "Max",
  age: 34,
  ville: "Nantes",
};
```

Si tu oublies `ville`, `tsc` proteste. Si tu mets `age: "trente"`, aussi. L'interface ne crée pas l'objet à l'exécution : elle guide le compilateur. Après compilation, il reste un objet JS normal.

★ A RETENIR

Une interface décrit la forme d'un objet. Tu la reutilises pour typer variables, paramètres et retours.

Ce que ce n'est pas

Ce n'est pas une classe (pas d'instances "new" obligatoires ici). Ce n'est pas une base de données. Ce n'est pas obligatoire pour un objet littéral unique jeté une fois. Ce n'est pas non plus `type` vs `interface` en débat infini : pour débuter, **interface** sur les objets suffit largement. Et ce n'est pas remplir vingt champs "au cas ou".

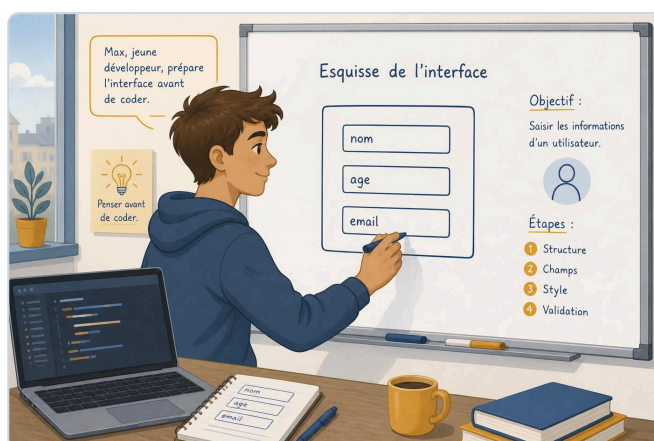
Propriétés et reemploi

```
interface Produit {  
  id: number;  
  label: string;  
  prix: number;  
}  
  
function afficherPrix(p: Produit): void {  
  console.log(p.label + " : " + p.prix + " EUR");  
}  
  
afficherPrix({ id: 1, label: "Joint", prix: 4.5 });
```

Lea passe des `Produit` à plusieurs fonctions sans recopier la forme. Max a commencé à documenter ses objets "todo" avec une interface : fini les `task.texte` vs `task.title` mélanges. Sam exige un nom d'interface qui parle : `TodoItem`, pas `Data` ou `Stuff`.

♦ ASTUCE

Nomme l'interface comme un nom commun clair : `Client`, `TodoItem`, `DevisLigne`. Evite `IData1`.



Exemple : Max dessine nom, age, email avant de coder.

Petite histoire

Lea et un stagiaire échangeaient un objet "commande" par message. Sans interface, chacun inventait des clés. Avec `interface Commande`, le stagiaire a vu tout de suite les champs manquants dans l'éditeur. La revue de code a duré dix minutes au lieu d'une heure de "ah oui j'avais mis `name` pas `nom`". Max a copié le geste sur sa todo artisan. DanielCraft adore ces gains invisibles : moins de théâtre, plus de livraison.

Erreur classique

Créer une interface puis passer un objet avec des clés en trop sans savoir si c'est autorisé (selon le contexte d'assignation, TS est strict sur l'excès pour les littéraux). Oublier une propriété obligatoire. Dupliquer la même forme dans cinq fichiers au lieu d'exporter. Autre piège : interface vide "pour plus tard". Préfère attendre d'avoir deux champs réels.

⚠ ATTENTION

Si un littéral object a une propriété inconnue pour l'interface, TypeScript peut refuser. C'est voulu : ça détecte les fautes de frappe (`emial` au lieu de `email`).

En vrai

Modèle une fiche `Livre` avec `titre`, `pages`, `lu` (boolean). Crée un objet conforme. Passe-le à une fonction `resume(l: Livre)`. Compile. Retire un champ, observe l'erreur, remets.

Faire vivre une interface

Une interface utile se lit comme une fiche. Tu dois pouvoir dire : "un `TodoItem`, c'est un id, un texte, et un flag fait". Si tu ne peux pas, simplifie. Lea interdit les interfaces de quinze champs pour un atelier d'une heure. Max a appris à sortir les détails rares en optionnel plutôt qu'à tout rendre obligatoire. Sam fait comparer deux interfaces mal nommées et bien nommées : la salle vote toujours pour la claire.

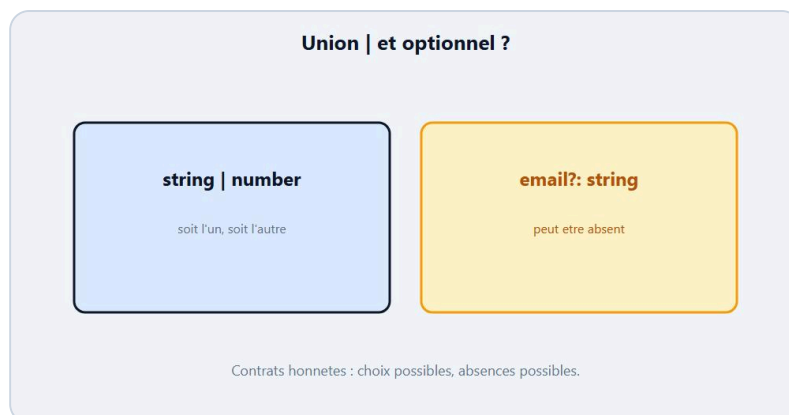
Tu peux réutiliser la même interface dans plusieurs fichiers plus tard (export/import). Pour ce livre, un seul fichier suffit. L'idée compte : une seule source de vérité pour la forme. Si tu changes `texte` en `title`, tu changes l'interface et `tsc` te montre tous les endroits à mettre à jour. C'est exactement le service rendu.

Évite aussi de confondre interface et valeur. L'interface ne "contient" pas les todos. Le tableau `TodoItem[]` contient les todos. L'interface dit seulement à quoi ressemble chaque élément. Chez DanielCraft, cette phrase revient souvent parce qu'elle évite une confusion durable.

A toi

Écris `interface TodoItem { id: number; texte: string; fait: boolean; }`. Crée deux todos. Écris `function marquerFait(t: TodoItem): void` qui met `fait` à `true` et log. Chez DanielCraft, cette interface reviendra dans les ateliers et le mini-projet.

Chapitre 6 - Unions et optionnels (| et ?)



Union | et optionnel ? : plusieurs cas possibles.

Parfois une valeur peut etre **soit** une chose **soit** une autre. C'est une **union** : `string | number`. Parfois une propriete peut etre absente. C'est l'**optionnel** : `email?: string`. Chez DanielCraft, ces deux outils evitent le mensonge du "toujours present, toujours le meme type". Lea type un id comme `number | string` quand l'API est capricieuse. Max marque `telephone?` sur ses contacts incomplets. Sam dit : "l'union dit les choix. Le point d'interrogation dit le droit a l'absence."

```
let id: number | string;
id = 12;
id = "abc-12";
// id = true; // erreur

interface Contact {
  nom: string;
  email?: string;
}

const a: Contact = { nom: "Lea" };
const b: Contact = { nom: "Max", email: "max@ex.com" };
```

Avant d'utiliser `email`, tu verifies qu'il existe. Avant de traiter `id` comme un nombre, tu verifies le type. C'est le pont vers le narrowing du chapitre 9. Sans ces deux outils, tu serais tente d'inventer des valeurs fantomes ("") juste pour "satisfaire" le type. Mieux vaut dire la verite au compilateur.

★ A RETENIR

| = plusieurs types possibles. ? = propriete facultative. Les deux rendent le contrat honnete.

Ce que ce n'est pas

Ce n'est pas une excuse pour `string | number | boolean | null | undefined | any`. Une union trop large ne protege plus. Ce n'est pas non plus ? partout "au cas ou" : trop d'optionnels = code plein de `if` defensifs. Et ce n'est pas confondre `email?: string` avec `email: string | undefined` (proches, nuances selon le contexte). Pour debuter, ? sur les champs vraiment facultatifs suffit. Lea limite volontairement ses unions a deux ou trois membres utiles.

Petits exemples utiles

```
type Statut = "brouillon" | "envoye" | "paye";

let s: Statut = "brouillon";
s = "paye";
// s = "annule"; // erreur si pas dans l'union

function longueur(x: string | string[]): number {
  return x.length; // ok : les deux ont length
}
```

Les unions de littéraux ("ok" | "ko") sont excellentes pour les états finis. Lea les préfère aux strings libres qui acceptent "OKK" par faute de frappe. Max a remplacé trois booléens confus par un statut clair. Sam fait lister les états au tableau avant d'ouvrir l'éditeur : le type écrit la réunion.

✦ ASTUCE

Pour un état métier (brouillon / envoye / paye), préfère une union de littéraux à une string ouverte.

Petite histoire

Sam a montré un formulaire où `age` arrivait parfois vide. Les élèves voulaient `age: number`. Il a proposé `age?: number` puis un test avant calcul. Lea, en prod, avait un bug parce qu'elle lisait `contact.email.toLowerCase()` sans guard. L'optionnel + un `if (contact.email)` a fermé le trou. Max a appliqué le même geste sur `notes?` de ses devis. DanielCraft résume : un type honnête vaut mieux qu'une valeur inventée pour faire plaisir au compilateur.

Erreur classique

Écrire `string | any` (inutile). Utiliser `?` puis accéder sans vérifier. Créer une union de dix types au lieu de modéliser mieux. Autre piège : optionnel sur un champ vraiment obligatoire métier, puis découvrir le trou en production. DanielCraft : le type doit raconter la réalité, pas la fantasmer.

⚠ ATTENTION

Si une propriété est `?`, vérifie avant d'appeler une méthode dessus. Sinon tu retrouves l'erreur runtime que TS essayait d'éviter.

En vrai

Modèle `interface Message { texte: string; auteur?: string }`. Crée deux messages, un avec auteur, un sans. Écris une fonction qui affiche `auteur` seulement s'il existe. Compile. Puis ajoute une union `priorite: "basse" | "haute"` et teste une mauvaise valeur.

Honnêter le contrat

Les unions et optionnels existent pour coller à la réalité. Un formulaire incomplet a des champs vides. Une API legacy renvoie parfois un id string. Mentir avec `email: string` alors qu'il manque souvent, c'est fabriquer des crashes. Lea préfère un `email?` + un message "email manquant" qu'un crash `.toLowerCase`. Max a mis trop d'`?` un temps ; son code était plein de branches. Sam l'a aidé à distinguer "rare mais

possible" et "toujours la".

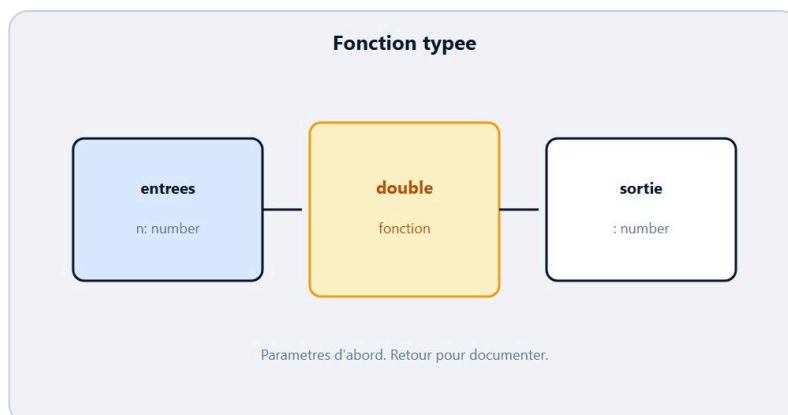
Pour les etats, les unions de litteraux brillent. `"brouillon" | "envoye" | "paye"` empeche `"payee"` avec accent foireux ou `"PAYE"`. Tu gagnes de l'autocompletion et tu perds des fautes. Combine avec un `switch` plus tard si besoin ; pour l'instant, des `if` suffisent.

Rappel : une union n'est pas un tableau. `string | number` est une valeur qui est l'un ou l'autre. `(string | number)[]` est une liste dont chaque case est l'un ou l'autre. Ce n'est pas la meme histoire. Lis a voix haute si tu bloques.

A toi

Type `let code: string | number`. Assigne les deux formes. Ecris `interface Ligne { label: string; quantite?: number }`. Cree trois lignes differentes. Chez DanielCraft, ce duo `|` et `?` revient sans cesse dans les APIs et formulaires. Note une phrase : "ou est-ce que mon code ment encore ?"

Chapitre 7 - Fonctions typees



Fonction typee : entrees et sortie annoncees.

Une **fonction** typee annonce le type de ses parametres et souvent le type de retour. Tu ecris `function double(n: number): number`. Le contrat est visible : entre un nombre, ressort un nombre. Chez DanielCraft, c'est le meilleur endroit pour annoter, parce que les bugs aiment se cacher dans les appels. Lea type toutes les entrees publiques. Max a compris `return` + type de retour le jour ou `tsc` a refuse un `return "ok"` dans une fonction `: number`. Sam separe encore `console.log` (effet) et `return` (valeur).

```
function direSalutA(prenom: string): void {
  console.log("Salut " + prenom);
}

function moyenne(a: number, b: number): number {
  return (a + b) / 2;
}

const m = moyenne(12, 16);
```

`void` signifie : pas de valeur utile renvoyee (souvent pour les fonctions qui affichent seulement). Sans annotation de retour, TypeScript infere souvent correctement. Explicite reste pedagogique. Quand tu partages une fonction avec quelqu'un d'autre, le type de retour lit comme une notice.

★ A RETENIR

Type les parametres d'abord. Le type de retour documente ce que l'appelant recoit.

Ce que ce n'est pas

Ce n'est pas obligatoire d'annoter le retour si l'inference est evidente - mais c'est utile pour apprendre. Ce n'est pas une fleche `=>` obligatoire : `function` suffit. Ce n'est pas `Function` (type trop large). Et ce n'est pas oublier d'appeler : une fonction typee non appelee ne fait toujours rien. Lea dit : "un contrat sans appel, c'est une promesse non tenue".

Parametres optionnels et defaults

```
function saluer(prenom: string, titre?: string): string {
  if (titre) {
    return titre + " " + prenom;
  }
  return prenom;
}

function clamp(n: number, max: number = 100): number {
  return n > max ? max : n;
}
```

Le `?` sur un parametre le rend facultatif. Une valeur par default (`= 100`) aussi, avec une nuance : le default fournit une vraie valeur. Lea utilise les defaults pour les seuils. Max prefere parfois deux fonctions courtes plutot qu'un parametre optionnel obscur. Sam rappelle l'ordre : parametres obligatoires d'abord, optionnels ensuite.

♦ ASTUCE

Si un parametre est souvent absent, donne un default clair. Si son absence change vraiment le comportement, garde `?` et un `if`.

Petite histoire

Lea avait `calculerTotal(lignes)` sans types. Un stagiaire passait une string. Le total devenait absurde. Avec `lignes: { prix: number; qte: number }[]` et un retour `number`, l'editeur a bloque l'appel foireux. Max a type `estMajeur(age: number): boolean` et a cesse de renvoyer `"oui"`. Sam applaudit les fonctions courtes : une mission, un retour clair. Chez DanielCraft, une signature lisible vaut un paragraphe de commentaire.

Erreur classique

Annoter le retour `number` puis `return` sans valeur (ou `return string`). Oublier le type d'un parametre et se retrouver avec `any` implicite selon la config. Donner trop de responsabilites a une seule fonction typee "usine". Autre piege : confondre `void` et `undefined`. Pour debuter, `void` = "je n'utilise pas le retour".

⚠ ATTENTION

Si tu annonces `: number` et que tu rates le `return`, TypeScript te le dit. Ecoute-le : c'est exactement le filet voulu.

En vrai

Ecris `aireRectangle(largeur: number, hauteur: number): number`. Teste. Retire un `return`, lis l'erreur, remets. Puis ajoute `estMajeur(age: number): boolean`. Appelle les deux depuis un petit `main` mental : entrees claires, sorties claires.

Signatures comme documentation

La signature d'une fonction typee est une doc courte qui ne ment pas. `moyenne(a: number, b: number): number` dit tout. Lea lit les signatures avant le corps en revue. Max a commence a ecrire la signature d'abord, puis le corps : ca l'oblige a clarir l'intention. Sam refuse les fonctions `faireTout(data: any): any` en copie.

Pense aussi aux fonctions pures vs effets. Une fonction qui calcule et `return` est facile a typer et a tester mentalement. Une fonction qui touche le DOM en plus melange les roles. Dans le mini-projet, `add` change l'etat, `render` affiche. Types simples des deux cotes. Chez DanielCraft, cette separation est une pratique autant qu'une affaire de types.

Si le retour est une union (`string | null`), documente-le. L'appelant saura qu'il doit narrowing. Cacher un `null` possible derriere un mensonge `: string` revient au chapitre optionnels : contrat deshonnete.

A toi

Ecris `function labelProduit(nom: string, prix: number): string` qui renvoie `"nom - prix EUR"`. Ecris `function logErreur(msg: string): void`. Appelle les deux. Note return vs void. Chez DanielCraft, ce reflexe porte tout le reste du livre. Si tu bloques, relis seulement les signatures avant le corps.

Chapitre 8 - Tableaux types

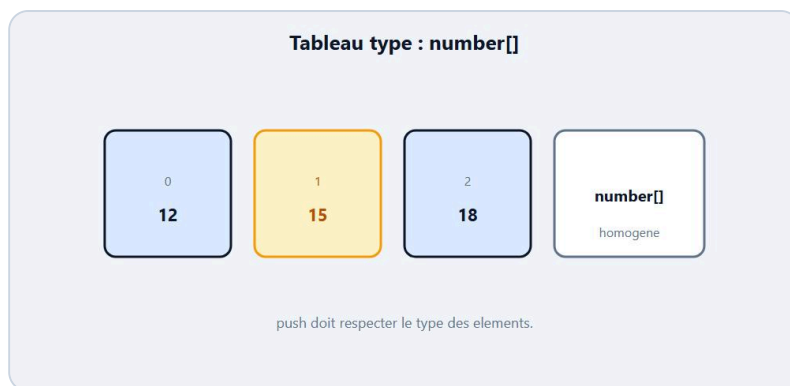


Tableau typee : une liste d'un meme contrat.

Un **tableau** type dit ce qu'il contient : `number[]` ou `Array<number>`. Les deux formes marchent ; `number[]` est courte et courante. Chez DanielCraft, on type les listes des le debut parce que c'est la que les melanges arrivent : un prix string au milieu de nombres, un todo sans `id`. Lea ecrit `lignes: DevisLigne[]`. Max a longtemps pousse n'importe quoi dans `let liste = []` ; en strict, TS demande souvent de preciser. Sam fait remplir un tableau homogene au tableau avant le code.

```
const notes: number[] = [12, 15, 18];
notes.push(14);
// notes.push("bien"); // erreur

const prenom: string[] = ["Lea", "Max", "Sam"];
```

Tu peux typer un tableau d'objets avec une interface :

```
interface TodoItem {
  id: number;
  texte: string;
  fait: boolean;
}

const todos: TodoItem[] = [
  { id: 1, texte: "Compiler", fait: true },
  { id: 2, texte: "Annoter", fait: false },
];
```

Une liste typee, c'est aussi de l'autocompletion : sur `todos[0]`, l'editeur propose `id`, `texte`, `fait`. Lea adore ce confort. Max a cesse de chercher "c'etait `title` ou `texte` ?" dans trois fichiers.

★ A RETENIR

`Type[]` = liste d'elements de ce type. Homogene autant que possible.

Ce que ce n'est pas

Ce n'est pas un objet (les index sont numeriques, ordres). Ce n'est pas une union magique `(string | number)[ ]` a utiliser partout : possible, mais souvent signe que le modele est flou. Ce n'est pas `any[ ]` "pour aller vite". Et ce n'est pas oublier que `push` doit respecter le type des elements. Sam dit : "si tu melanges, c'est souvent deux listes qui se cachent".

Lire et transformer

```
function moyenne(notes: number[]): number {
  if (notes.length === 0) {
    return 0;
  }
  let somme = 0;
  for (const n of notes) {
    somme += n;
  }
  return somme / notes.length;
}

const m = moyenne([10, 12, 14]);
```

Lea type le parametre `notes: number[ ]` et le retour. Max a decouvert que `todos.filter(...)` garde souvent un type coherent. Sam insiste sur le cas liste vide : decide un comportement (0, erreur, message) au lieu de laisser `NaN`. Chez DanielCraft, gerer le vide fait partie du typage autant que le `[ ]` lui-meme.

♦ ASTUCE

Si ta liste peut etre vide, gere `length === 0` avant de diviser ou de prendre `[0]`.

Petite histoire

Max melangeait ids number et string dans un meme tableau "parce que l'API...". En TS, il a choisi `string[ ]` et normalise tout en string a l'entree. Moins heroique, plus stable. Lea a type `Produit[ ]` et a vu l'autocompletion proposer `prix` partout. Sam montre aux eleves l'erreur `push` d'un mauvais type : le "ah" collectif revient a chaque promo. Une seule erreur de `push`, et le message devient pedagogique.

Erreur classique

Declarer `let items = [ ]` sans type puis empiler des formes differentes. Utiliser `Array` sans parametre. Croire que `todos[0]` est toujours defini (il peut etre `undefined` si vide). Autre piege : tableau de `any` pour "passer". DanielCraft refuse ce raccourci en formation. Prefere annoter des le `const`.

⚠ ATTENTION

Acceder a `liste[0]` sur une liste peut-etre vide : verifie la longueur ou assume le risque consciemment.

En vrai

Cree `const prix: number[] = [4.5, 10, 2]`. Calcule la somme dans une fonction typee. Ajoute un produit via `push`. Tente un `push` de string, lis l'erreur. Puis filtre les prix superieurs a 5 et observe que le resultat reste `number[]`.

Listes et coherence

Une liste typee protege surtout la coherence des elements. Si tu pushes un objet incomplet dans `TodoItem[]`, `tsc` rale tout de suite. Lea adore ca en atelier : le stagiaire voit l'oubli `fait` avant la demo. Max poussait des shapes differentes "temporairement" ; le temporaire restait. Sam impose un seul type d'element par liste debutante.

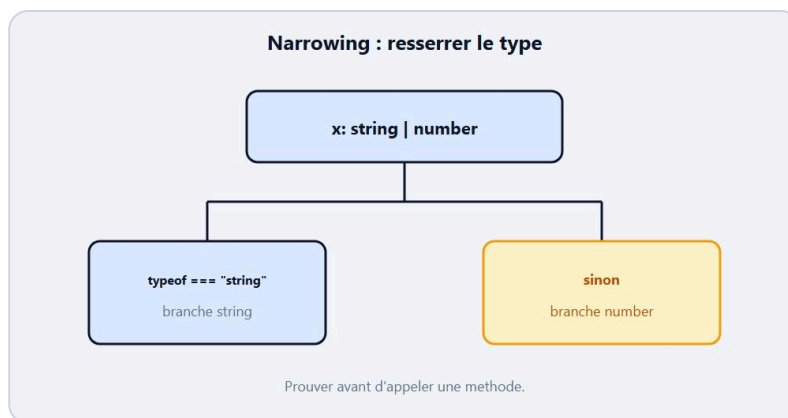
Tu croiseras `readonly Type[]` plus tard pour interdire `push`. Pas besoin maintenant. Tu croiseras aussi les tuples `[string, number]`. Pas besoin non plus pour un compteur ou une todo. Reste sur `Type[]` tant que ta liste est vraiment une liste ouverte.

Quand tu filtres, le type reste souvent `Type[]`. Quand tu maps vers autre chose, le type de sortie change : `todos.map(t => t.texte)` donne `string[]`. Observe l'inference. Si elle te surprend, annote le resultat. Chez DanielCraft, observer l'inference est une competence, pas de la paresse.

A toi

Modele `TodoItem[]` avec trois taches. Ecris `function restantes(todos: TodoItem[]): TodoItem[]` qui renvoie celles ou `fait === false`. Affiche le resultat. Chez DanielCraft, ce pattern liste + filtre revient dans le mini-projet. Garde le fichier pour le chapitre 13.

Chapitre 9 - Narrowing (resserrer le type)



Narrowing : le type se precise dans un if.

Le **narrowing**, c'est quand TypeScript comprend qu'à cet endroit du code, une union est devenue un type plus précis. Tu testes avec `typeof`, avec `if (valeur)`, avec une comparaison, et dans la branche le type se resserre. Chez DanielCraft, on présente ça comme un entonnoir : large à l'entrée, étroit après la garde. Lea écrit `if (typeof id === "string")` avant d'appeler `.toUpperCase()`. Max a longtemps ignoré les guards et forçait avec `as`. Sam refuse les `as` débutants : "prouve d'abord".

```
function labelId(id: string | number): string {
  if (typeof id === "string") {
    return id.toUpperCase(); // ici : string
  }
  return "N-" + id; // ici : number
}
```

Après le `typeof === "string"`, TS sait que tu as une string. Dans le `else`, il reste le number. Tu n'as pas menti. Tu as filtré. Ce geste est le compagnon naturel des unions et des optionnels : sans lui, tu retombes dans le forçage ou dans le crash runtime.

★ A RETENIR

Narrowing = prouver le type dans un `if` (souvent avec `typeof`) pour que TS autorise les opérations sûres.

Ce que ce n'est pas

Ce n'est pas de la magie runtime spéciale TypeScript : ce sont des tests JS classiques que le compilateur lit. Ce n'est pas `as Type` (assertion). Ce n'est pas non plus obligatoire partout : si le type est déjà précis, pas besoin. Et ce n'est pas un chapitre "avance interdit débutants" : c'est le geste quotidien des unions. Lea narrowing sans le nommer parfois : elle dit juste "je vérifie avant".

Guards utiles pour debuter

```
function afficherEmail(email: string | undefined): void {
  if (!email) {
    console.log("(pas d'email)");
    return;
  }
  console.log(email.toLowerCase()); // string
}

function taille(x: string | string[]): number {
  if (Array.isArray(x)) {
    return x.length;
  }
  return x.length; // string aussi a length, mais tu vois le pattern
}
```

Pour les optionnels, un simple `if (valeur)` narrow souvent vers la presence. Pour les objets, tu croieras plus tard les type guards personnalisés ; ici, `typeof` et tests de presence suffisent. Max a appris `Array.isArray` le jour ou une union string/tableau l'a bloqué dix minutes.

♦ ASTUCE

Quand `tsc` dit qu'une propriete n'existe pas sur l'union, ajoute un `if / typeof` au lieu d'un `as`.

Petite histoire

Lea recevait `reponse: string | number` d'un vieux formulaire. Sans narrowing, `.trim()` cassait sur les nombres. Avec `typeof`, deux branches propres. Max affichait `contact.email` optionnel ; un `if (contact.email)` a fait disparaître l'erreur et le crash. Sam projette le message d'erreur "Property does not exist" puis le `if` qui le soigne. La salle retient mieux que le jargon "narrowing". DanielCraft celebre le `if` qui prouve.

Erreur classique

Utiliser `as string` pour faire taire l'erreur sans tester. Tester trop tard (apres l'appel dangereux). Oublier le `return` dans la branche d'echec et laisser TS confus. Autre piege : `typeof null` (curiosite JS) - pour debuter, prefere des unions claires et des tests de presence.

⚠ ATTENTION

`as` force. Narrowing prouve. Prefere prouver. Le forçage revient te mordre au runtime.

En vrai

Ecris `function describe(x: string | number)` qui log "texte: ..." ou "nombre: ...". Utilise `typeof`. Compile. Retire le `if`, observe l'erreur sur une methode string, remets. Note la difference de message : c'est ton professeur.

Prouver avant d'agir

Le narrowing est le geste anti-crash par excellence. Tu ne demandes pas a TypeScript de "croire". Tu lui montres un test que JavaScript fera vraiment. `typeof`, `===`, `in`, `Array.isArray`, un simple `if (valeur)` : autant de preuves. Lea refuse les PR ou un `as` remplace un `if` de trois lignes. Max a gagne en confiance le jour ou il a ecrit un narrowing sans y penser. Sam fait effacer le `if` en live pour revoir l'erreur revenir : pedagogie brutale et efficace.

Attention aux fausses preuves. Un test trop large ne resserre pas. Un test apres l'usage dangereux arrive trop tard. Place le garde avant l'appel de methode. Structure souvent en early return : si invalide, message et `return` ; ensuite le type est sur.

Ce chapitre se relie a `unknown` : tu narrowing pour sortir de l'inconnu. Il se relie aux optionnels : tu narrowing pour prouver la presence. Une seule idee sous plusieurs syntaxes.

A toi

Prends `interface User { nom: string; age?: number }`. Ecris `function ageLabel(u: User): string` qui renvoie `"age inconnu"` ou `"N ans"` apres narrowing sur `u.age`. Chez DanielCraft, ce reflexe est le compagnon naturel des unions. Si tu reussis, tu es pret pour `unknown` au chapitre suivant.

Chapitre 10 - any et unknown



any coupe les alarmes. unknown demande de verifier.

any desactive le typage pour une valeur : TypeScript arrete de verifier. C'est pratique pour "faire taire" une erreur, et dangereux pour la meme raison. **unknown** dit : je ne sais pas encore, tu devras prouver avant d'utiliser. Chez DanielCraft, on enseigne **any** comme une sortie de secours rare, pas comme un style de vie. Lea le bannit des APIs internes. Max a mis **any** partout un week-end et a retrouve les bugs JS qu'il fuyait. Sam ecrit au tableau : "**any** = je renonce. **unknown** = je reporterai la preuve."

```
let souple: any = "salut";
souple = 12;
souple.foo.bar; // TS ne proteste pas - crash possible au runtime

let prudent: unknown = "salut";
// prudent.toUpperCase(); // erreur : il faut narrowing
if (typeof prudent === "string") {
  console.log(prudent.toUpperCase());
}
```

Le contraste est volontairement brutal. Avec **any**, le filet tombe. Avec **unknown**, le filet reste, mais tu dois passer le garde. Si tu viens du narrowing, tu as deja le reflexe.

★ A RETENIR

Evite **any**. Prefere un vrai type, une union, ou **unknown** + narrowing si tu ne sais pas encore.

Ce que ce n'est pas

Ce n'est pas "interdit a vie" : un proto jetable peut avoir un **any** temporaire. Ce n'est pas non plus **unknown** partout par snobisme. Ce n'est pas **Object** comme substitut magique. Et ce n'est pas laisser **noImplicitAny** off pour ne jamais apprendre. En mode **strict**, TS te pousse a preciser - c'est voulu. Lea accepte un **any** local avec un **TODO** date, pas un **any** dans une interface partagee.

Quand tu touches une donnée floue

```
function lireTitre(data: unknown): string {
  if (
    typeof data === "object" &&
    data !== null &&
    "titre" in data &&
    typeof (data as { titre: unknown }).titre === "string"
  ) {
    return (data as { titre: string }).titre;
  }
  return "(sans titre)";
}
```

Pour debuter, tu n'as pas besoin de ce niveau de garde complet. L'idée compte : `unknown` t'oblige à demander. Lea simplifie souvent avec une interface des qu'elle connaît la forme. Max parse du JSON (`JSON.parse` renvoie souvent `any` selon le contexte) puis valide les champs un à un. Sam préfère "valider puis typer" à "typer puis prier".

⚠ ATTENTION

`JSON.parse` ne "devine" pas ton interface. Valide ou annote après preuve. Un `as MonType` naïf sur du JSON est un `any` déguisé.

Petite histoire

Un stagiaire de Lea avait mis `any` sur toute la réponse HTTP. La demo a planté sur `undefined.toFixed`. Elle a remplacé par une interface `ApiDevis` + un check minimal. Max a cherché "comment désactiver TypeScript" après trois erreurs ; Sam lui a montré comment les lire. Le lendemain, Max utilisait `unknown` sur une entrée utilisateur et était fier du `typeof`. DanielCraft célèbre ce basculement : du silence acheté au filet choisi.

Erreur classique

`any` pour faire compiler la CI. `any` copie-collé dans une interface. Croire que `unknown` est inutilisable (il l'est, avec narrowing). Autre piège : `as any` en fin de ligne "juste pour aujourd'hui" qui reste six mois. Mets un commentaire `// TODO typer` si tu dois vraiment, et reviens.

♦ ASTUCE

Si tu tapes `any`, demande-toi : "quelle union ou interface dirait la même chose en plus honnête ?"

En vrai

Déclare `let x: any` et appelle une méthode inventée : observe le silence de `tsc`. Remplace par `unknown`, ajoute un `typeof`, vois la différence. Garde le second réflexe. Puis tente `JSON.parse('{ "a":1 }')` et refuse de le traiter comme interface sans check.

Sortir du flou sans mentir

Le vrai travail avec des données floues, c'est la validation. Tu reçois un JSON, tu vérifies les champs, tu construis un objet type. Lea écrit parfois une fonction `parseClient(data: unknown): Client | null`. Max a voulu un seul `as Client` ; ça a cassé en prod sur un champ renommé. Sam montre les deux versions côte à côte : le parse est plus long, le crash disparaît.

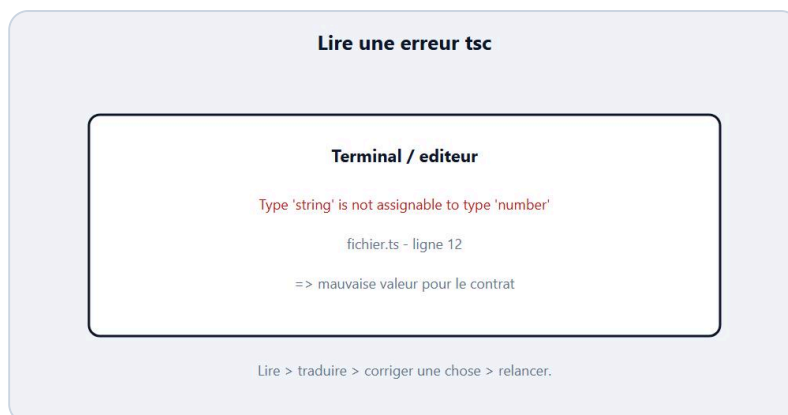
`any` reste tentant dans les callbacks obscurs ou les libs mal typées. Isole-le. Ne le laisse pas contaminer toute ton appli : une variable `any` infecte souvent ce qu'elle touche. Préfère typer le résultat des que tu contrôles la forme.

Enfin, rappelle-toi le but du livre : coder plus sûr. `any` annule ce but localement. Si tu l'utilises, sache que tu es repassé en mode JS nu à cet endroit. Parfois ok. Souvent non.

A toi

Écris une fonction `toNumber(v: unknown): number | null` qui renvoie le nombre si `typeof v === "number"`, sinon `null`. Teste avec `12`, `"12"`, `true`. Chez DanielCraft, cette fonction est un modèle mental anti-`any`. Si `"12"` te démange, ajoute une branche `parseFloat` consciente - pas un `as number`.

Chapitre 11 - Lire une erreur du compilateur



Lire une erreur tsc : message, ligne, corriger.

Une erreur **tsc**, ce n'est pas une insulte. C'est un rapport : fichier, ligne, message. Chez DanielCraft, on apprend à lire avant de googler au hasard. Lea ouvre toujours le premier diagnostic, pas le vingtième. Max respirait mal devant le rouge ; maintenant il cherche le type attendu vs le type fourni. Sam enseigne une grille : ou ? quoi ? pourquoi probable ? correction minimale ?

Le message typique ressemble à : `Type 'string' is not assignable to type 'number'`. Traduction : tu ranges du texte dans une boîte nombre. Autre classique : `Property 'email' does not exist on type 'Contact'`. Traduction : soit faute de frappe, soit champ absent de l'interface, soit narrowing manquant. Encore : `Object is possibly 'undefined'`. Traduction : optionnel ou tableau vide, ajoute un garde. Une fois que tu traduis, la peur baisse.

★ A RETENIR

Lis l'erreur comme une phrase : type fourni vs type attendu, propriété manquante, valeur peut-être vide. Puis corrige une chose.

Ce que ce n'est pas

Ce n'est pas "TypeScript est cassé". Ce n'est pas une raison d'ajouter `as any`. Ce n'est pas ignorer la ligne indiquée pour réécrire tout le fichier. Et ce n'est pas honteux d'avoir des erreurs : c'est le mode normal d'apprentissage. Lea en produit encore. La différence, c'est la vitesse de lecture. Max notait autrefois "ça marche pas" ; maintenant il note le message exact.

Méthode calme

1. Ouvre le fichier et la ligne cibles.
2. Lis le message jusqu'au bout (souvent la fin dit l'attendu).
3. Identifie si c'est assignation, appel de fonction, accès propriété, ou absence de return.
4. Corrige le minimum : type, valeur, `if`, ou signature.
5. Relance **tsc**. Une erreur de moins = progrès.

```
let total: number = 0;
// total = "120";
// error: Type 'string' is not assignable to type 'number'.
total = 120;
```

Parfois l'editeur souligne en rouge avant meme `tsc`. C'est le meme moteur. Lea regarde le survol (hover) : TypeScript explique souvent l'attendu. Sam demande aux eleves de lire a voix haute : la salle rit, puis comprend.

✦ ASTUCE

Corrige la premiere erreur d'abord. Les suivantes sont parfois des cascades.



Exemple : Sam lit l'erreur au calme avant de patcher.

Petite histoire

Max a passe une soiree a "desactiver strict" parce qu'une propriete optionnelle le genait. Sam lui a montre le `if (user.age !== undefined)`. Dix minutes. Lea, en revue, refuse les PR qui ajoutent `// @ts-ignore` sans justification. Elle demande le message original dans le commentaire de revue. DanielCraft : le rouge est un professeur severe mais coherent. Le jour ou Max a corrige trois erreurs sans chat, il a sourit tout seul.

Erreur classique

Lire seulement "error" et paniquer. Coller le message dans un chat sans le fichier. Ajouter `!` (non-null assertion) partout pour silence. Autre piege : corriger le type pour qu'il accepte n'importe quoi (`string | number | boolean | null`) au lieu de corriger la donnee. Prefere la donnee honnete.

⚠ ATTENTION

`// @ts-ignore` et `as any` font taire le professeur. Ils n'enlevent pas le bug. Utilise-les seulement en dernier recours documente.

En vrai

Introduis volontairement trois erreurs : mauvaise assignation, propriété inconnue, return manquant. Lis chaque message à voix haute. Corrige. Tu entraînes le muscle. Chronomètre-toi : le but n'est pas la vitesse olympique, c'est la lecture complète.

Traduire le rouge en action

Fabrique-toi un mini dictionnaire perso. "not assignable" = mauvaise valeur pour ce type. "does not exist" = champ inconnu ou narrowing manque. "possibly undefined / null" = garde manquant. "Expected N arguments" = appel de fonction incomplet. Lea colle ce dictionnaire dans le README stagiaires. Max l'a dans sa tête maintenant. Sam interroge : "traduis cette erreur en une phrase française" avant d'autoriser la correction.

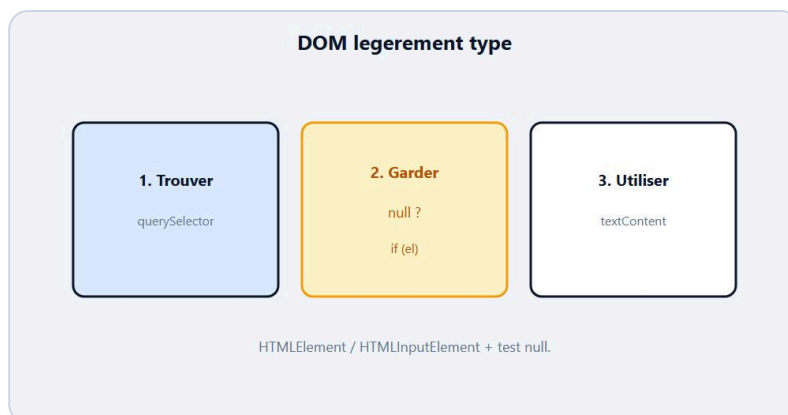
Ne corrige pas au hasard. Change une chose, recompile, observe. Si l'erreur se déplace, tu progresses. Si elle mute en autre chose, lis la nouvelle. Les cascades existent : une mauvaise interface produit dix diagnostics. Repare la source.

Évite aussi le piège du message en anglais "je ne comprends rien". Tu n'as pas besoin de tout comprendre : tu as besoin des mots types et des noms de variables cités. Relie-les à ton code. C'est un jeu de pistes, pas une dissertation.

A toi

Prends un petit fichier type. Casse-le. Note pour chaque erreur : (1) ligne, (2) en français ce que TS reproche, (3) ta correction. Chez DanielCraft, cette fiche d'erreurs vaut un chapitre de théorie supplémentaire. Garde-la à côté du mini-projet.

Chapitre 12 - DOM legerement type



DOM typee : element HTML avec des gardes simples.

Tu as deja selectionne des elements en JavaScript : `document.querySelector`, `getElementById`, `textContent`. En TypeScript, le DOM reste le meme, mais les types te rappellent qu'un element peut etre **absent** (`null`) et qu'un element generique n'a pas toujours les proprietes d'un `HTMLInputElement`. Chez DanielCraft, on reste leger : pas de framework, juste des gardes simples. Lea type ses boutons apres les avoir trouves. Max oubliait le `null` et voyait `Object is possibly 'null'`. Sam insiste : "trouve, verifie, utilise".

```
const btn = document.querySelector("#plus");
if (btn) {
  btn.addEventListener("click", () => {
    console.log("clac");
  });
}
```

Souvent `querySelector` renvoie `Element` | `null`. Si tu as besoin d'un champ `value`, tu vises un input :

```
const input = document.querySelector<HTMLInputElement>("#nom");
if (input) {
  console.log(input.value);
}
```

Le generique `<HTMLInputElement>` dit a TS quelle forme tu attends. Ca ne garantit pas que le HTML est juste : si l'id pointe un `div`, tu auras un souci runtime. Lea verifie aussi le HTML. Max a appris a lire `null` comme "pas trouve", pas comme "TypeScript est mechante".

★ A RETENIR

DOM type = selection + garde `null` (+ type d'element si tu lis `value`, etc.).

Ce que ce n'est pas

Ce n'est pas un cours HTML complet. Ce n'est pas React. Ce n'est pas `document.getElementById!` partout avec assertion non-null. Ce n'est pas non plus typer tout le document. Et ce n'est pas paniquer si TS te demande un `if` : c'est exactement le filet utile sur une page incomplete.

Gestes du quotidien

```
const scoreEl = document.querySelector<HTMLElement>("#score");
const plus = document.querySelector<HTMLButtonElement>("#plus");

function afficher(n: number): void {
  if (!scoreEl) return;
  scoreEl.textContent = String(n);
}

plus?.addEventListener("click", () => {
  // logique compteur plus loin
});
```

L'opérateur `?.` (optionnel) évite d'écouter un bouton absent. Lea l'aime pour les scripts de demo. Max préfère un `if (plus)` explicite au début. Sam accepte les deux si le garde existe vraiment.

♦ ASTUCE

Si tu lis `.value`, vise `HTMLInputElement` (ou `HTMLTextAreaElement`). Si tu changes seulement le texte, `HTMLElement` suffit souvent.

Petite histoire

Lea branchait un compteur. Sans garde, un mauvais id plantait la page. Avec `if (!scoreEl) return`, le script restait silencieux et elle voyait tout de suite l'id manqué dans le HTML. Max a forcé `as HTMLInputElement` sur un `span` : `value` était `undefined` à l'exécution. Sam a effacé le `as` et ajouté le bon sélecteur. DanielCraft : le type suit le HTML, il ne le remplace pas.

Erreur classique

Ignorer `null`. Utiliser `!` après chaque query "pour compiler". Croire que le générique `querySelector<T>` valide le DOM. Autre piège : tout mettre en `any` dès que le DOM résiste. Préfère un `if` et un type précis.

⚠ ATTENTION

`querySelector<HTMLInputElement>` ne vérifie pas le HTML. Il change seulement ce que TypeScript croit. Aligne id et balise.

En vrai

Crée une page minimale avec un `span#score` et un `button#plus`. Sélectionne les deux en TS, garde contre `null`, change le texte au clic. Compile. Retire un id, observe le comportement de ton garde.

DOM et types : le duo

Le DOM est vivant : l'utilisateur clique, le HTML peut être incomplet, un id peut manquer. TypeScript ne remplace pas ces réalités. Il te force à les admettre. Lea regarde toujours le HTML en même temps que le TS. Max ouvrait seulement le `.ts` et perdait du temps. Sam projette les deux fichiers. Chez DanielCraft, "aligner les ids" est une pratique autant qu'une annotation.

Pour les evenements, tu verras parfois `Event`, `MouseEvent`. Au debut, une fleche `() => add(1)` suffit sans typer l'event. Si tu as besoin de `event.target`, tu narrowing ou tu cast avec prudence vers un element connu. Reste simple : le compteur n'a pas besoin d'une these sur les events.

Si tu compiles vers un JS charge en `defer` / fin de body, le DOM existe souvent au demarrage du script. Si tu charges trop tot, `getElementById` renvoie null. Encore une fois : le type te le rappelle, le HTML/timing decide.

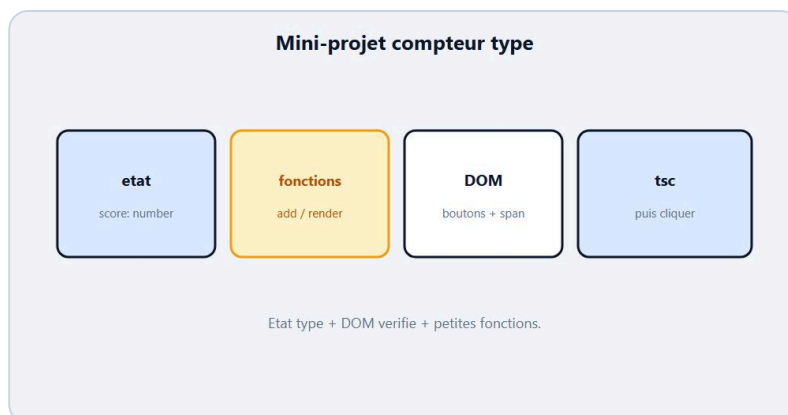
Lien avec le livre JS

En JavaScript bases, tu selectionnais et tu ecoutais des clics. Ici, tu fais pareil avec un filet. Le `querySelector` peut etre null : TS te le rappelle. `value` existe sur un `input` : TS te pousse vers `HTMLInputElement`. Rien de magique, juste plus de franchise. Lea dit que TS rend visibles des pieges que JS laissait silencieux. Max confirme apres son premier null. Sam en fait le pont officiel entre les deux livres.

A toi

Ecris un script qui lit un `input#prenom` et ecrit "Bonjour X" dans un `p#out` au clic d'un bouton. Types + gardes. Chez DanielCraft, ce micro-DOM prepare le mini-projet compteur.

Chapitre 13 - Mini-projet : compteur type



Mini-projet : etat type + actions claires.

Il est temps d'assembler. Tu vas construire un **compteur** type : un etat **number**, des actions claires (**incrémenter**, **reset**), un affichage DOM garde. Chez DanielCraft, un mini-projet n'est pas un portfolio. C'est une preuve que les briques tiennent ensemble. Lea livre souvent ce genre de demo en formation. Max a fini le sien un dimanche. Sam chronometre : "deux heures max pour la V1".

L'idée : un score commence a 0. Un bouton ajoute 1. Un autre remet a 0. Le span affiche la valeur. Tout est annote. Pas de **any**. Pas de **as** gratuits. Si **tsc** proteste, tu lis et tu corriges.

★ A RETENIR

Etat type + fonctions typees + DOM garde = un mini-projet TypeScript digne de ce nom.

Ce que ce n'est pas

Ce n'est pas une app complete. Ce n'est pas Redux. Ce n'est pas "il faut Vite". Un fichier HTML + un fichier **.ts** compile suffisent. Ce n'est pas non plus une todo entiere (tu la feras en atelier). Ici, le compteur force l'essentiel : nombre, void, null check.

Squelette

```
let score: number = 0;

const scoreEl = document.querySelector<HTMLElement>("#score");
const btnPlus = document.querySelector<HTMLButtonElement>("#plus");
const btnReset = document.querySelector<HTMLButtonElement>("#reset");

function afficher(): void {
  if (!scoreEl) return;
  scoreEl.textContent = String(score);
}

function incrementer(): void {
  score = score + 1;
  afficher();
}

function reset(): void {
  score = 0;
}
```

```

    afficher();
}

btnPlus?.addEventListener("click", incrementer);
btnReset?.addEventListener("click", reset);
afficher();

```

HTML minimal : un span, deux boutons, le script compile. Lea separe volontairement `afficher` du reste : une fonction, un role. Max a d'abord tout mis dans le listener ; Sam l'a fait extraire.

♦ ASTUCE

Compile souvent. Une erreur apres dix lignes se trouve plus vite qu'apres deux cents.



Exemple : Lea, Max et Sam valident le compteur type.

Petite histoire

Lea, Max et Sam ont valide le compteur ensemble. Lea a refuse un `score: any` . Max a oublie le garde sur `scoreEl` ; `tsc` l'a rattrape en strict. Sam a demande un `reset` type `void` et un affichage initial. En vingt minutes, le trio avait une demo stable. DanielCraft garde ce rythme : petit, clair, testable. Pas de ceremony.

Erreur classique

Laisser `score` en string parce que `textContent` est string. Melanger logique et DOM dans une seule fonction geante. Oublier d'appeler `afficher` apres mutation. Autre piege : compiler une fois, puis ne plus relancer `tsc` pendant une heure. Le filet ne sert que si tu le tendes.

⚠ ATTENTION

`textContent` est string. Ton etat peut rester `number` : convertis a l'affichage avec `String(score)` .

En vrai

Code le compteur. Ajoute un bouton `-1` qui ne descend pas sous 0. Type tout. Si tu bloques sur le DOM, relis le chapitre 12. Si tu bloques sur le type de `score` , relis les annotations.

Construire sans te perdre

Decoupe le travail en tranches. D'abord l'etat type et `render` avec un score fixe. Ensuite un bouton +. Ensuite moins et reset. Ensuite les gardes null. Enfin une regle metier (plafond, message). Lea travaille exactement dans cet ordre avec les clients presses. Max voulait tout d'un coup et se perdait dans cinq erreurs. Sam impose des commits mentaux : "ca compile ?" entre chaque tranche.

Garde le CSS minimal. Une page laide qui clique bat une page belle qui ne compile pas. Tu pourras habiller apres. Le livre JavaScript bases t'a deja montre le compteur en JS : ici tu ajoutes le filet. Compare les deux versions si tu les as. Tu verras que la logique est la meme, le contrat est plus visible.

Quand tu bloques, relis le chapitre erreur compilateur. Quand le clic ne fait rien, verifie le JS charge et les ids. Quand le type gene, demande si c'est le type ou la valeur qui ment. Chez DanielCraft, ces trois portes resolvent presque tous les blocages debutants.

Checklist avant de montrer

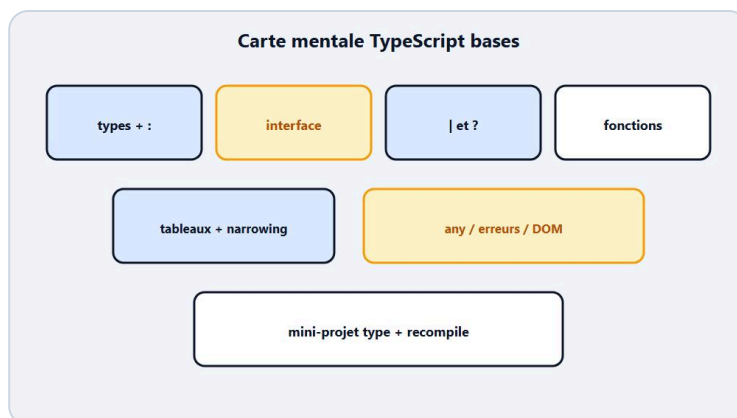
Coches mentales : score en number, render appele apres chaque changement, boutons trouves (ou message clair si absents), tsc sans erreur, page qui charge le bon js. Lea lit cette liste a voix haute avant un appel client. Max l'a scraptee sur un sticky. Sam met un point si un eleve montre sans avoir recompilé.

Si tu choisis la variante todo, ta checklist change un peu : interface presente, tableau type, trim sur la saisie, liste DOM reconstruite proprement. Dans les deux cas, zero any, une extension personnelle, un sourire a la fin. Chez DanielCraft, le sourire compte : il dit que tu as traverse le rouge sans abandonner.

A toi

Livre ta V1. Puis ecris en trois lignes : (1) ce que `tsc` a refuse, (2) comment tu as corrige, (3) ce que tu ajouterais demain (plafond, pas de score negatif deja fait...). Chez DanielCraft, cette note de fin transforme l'exercice en apprentissage.

Chapitre 14 - A retenir



Types, interfaces, unions, erreurs : la carte.

Tu as traversé le socle TypeScript débutant. Ce chapitre ne rajoute presque rien de neuf : il range. Chez DanielCraft, on fait souvent une carte avant les ateliers. Lea coche ce qu'elle réutilise chaque semaine. Max relit cette page avant une session de code. Sam s'en sert comme checklist orale en fin de module.

TypeScript, c'est JavaScript plus des **types** vérifiés avant l'exécution. Tu écris en `.ts`, tu lances `tsc`, tu obtiens du `.js`. Les annotations (`: string`) et les **interfaces** décrivent des contrats. Les **unions** (`|`) et les **optionnels** (`?`) disent la vérité sur les cas multiples. Les fonctions et tableaux types rendent les appels plus sûrs. Le **narrowing** prouve un type dans un `if`. Tu évites `any`, tu préfères `unknown` ou un vrai modèle. Tu lis les erreurs du compilateur au calme. Tu gardes le DOM avec des `null` checks.

★ A RETENIR

Types, interfaces, unions, narrowing, erreurs lues : voilà la carte. Le reste est de la pratique.

Ce que ce n'est pas

Ce n'est pas la fin de TypeScript. Generics avancés, utilitaires `Partial`, monorepos, decorateurs : hors scope ici. Ce n'est pas non plus un examen. Si une case de la carte est floue, tu retournes au chapitre, tu ne fais pas semblant. Lea dit : "mieux vaut trois briques solides que vingt notions citées".

Carte rapide

- Install / `tsc` / `tsconfig` léger
- `string`, `number`, `boolean`
- annotation `: type` et inférence
- interface pour les objets
- `|` et `?`
- fonctions typées (params + retour)
- `Type[]`
- narrowing avec `typeof` / `if`
- `any` rare, `unknown` + preuve
- lire `tsc`, pas l'éteindre

- DOM : query + garde
- mini-projet compteur

Max imprime parfois cette liste. Sam la fait reformuler a voix haute sans jargon. DanielCraft mesure le succes a "je peux expliquer a un ami", pas a "j'ai survole vingt videos".

♦ ASTUCE

Pour chaque item de la carte, cite un exemple de ton propre code. Si tu ne peux pas, relis le chapitre correspondant.

Petite histoire

Apres le mini-projet, Max a dit "en fait c'est surtout de la precision". Lea a sourit : c'etait exactement le message. Sam a ferme le projecteur et demande trois mots. Les eleves ont repondu : contrat, compilateur, preuve. Pas "framework". Pas "magie". Chez DanielCraft, ces trois mots suffisent pour repartir coder.

Erreur classique

Croire qu'il faut tout memoriser avant d'ouvrir un fichier. Ou au contraire tout sauter pour "faire React demain". Autre piege : garder `any` "en attendant" sans date de retour. La carte sert a prioriser, pas a culpabiliser.

⚠ ATTENTION

Si tu ne retiens qu'une chose : lis l'erreur, corrige le contrat, recompile. Ce cycle bat la theorie isolee.

En vrai

Sans notes, ecris sur papier les dix idees que tu gardes. Compare a la liste ci-dessus. Entoure les trous. Planifie deux relectures cibles avant les ateliers.

Fil rouge DanielCraft

Si tu ne devais garder que cinq phrases : (1) TS verifie avant d'executer. (2) Annoter le coeur. (3) Interface pour les objets. (4) Prouver avec narrowing. (5) Eviter `any`, lire `tsc`, recompiler. Tout le reste du livre illustre ces phrases avec Lea, Max et Sam.

Relie aussi mentalement ce livre au parcours JavaScript bases. Variables, fonctions, tableaux, DOM : tu les as deja. TypeScript ne les remplace pas. Il les etiquette. Quand tu reviendras a un script JS nu, tu sentiras le manque du filet - et tu sauras quoi ajouter.

Prochaine etape pratique : les ateliers. Ils ne rajoutent pas beaucoup de theorie. Ils forcent le geste. Fais-les pour de vrai, meme imparfaits. Un atelier termine vaut trois chapitres relu a vitesse x2.

Reviser en boucle courte

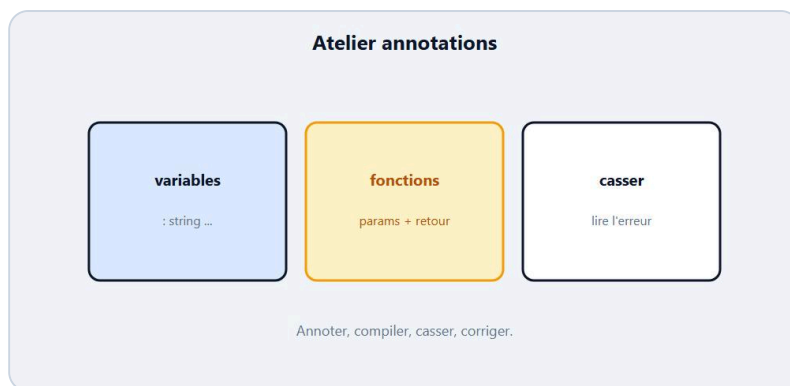
Prends trois soirs de vingt minutes. Soir 1 : annotations + fonctions. Soir 2 : interfaces + unions + narrowing. Soir 3 : DOM + mini projet + lecture d'erreurs. Tu n'as pas besoin de tout relire. Tu as besoin de refaire. Lea revise comme ca avant de former. Max a prefere binge-watch des videos et a moins retenu. Sam impose la boucle courte en fin de module.

Garde aussi une question talisman : quelle valeur illégale mon type refuse-t-il ? Si tu ne sais pas répondre pour une variable, ton annotation est décorative. Si tu sais, tu pilotes. C'est le test DanielCraft pour savoir si tu as vraiment fini les bases.

A toi

Choisis un petit script JS personnel. Liste trois endroits où un type t'aiderait. Annote-les en TS. Compile. Tu viens de transformer la carte en geste. C'est le but DanielCraft.

Chapitre 15 - Atelier : annoter sans paniquer



Atelier : annoter sans paniquer.

Objectif : prendre un petit bout de JavaScript flou et le rendre honnête avec des **annotations**. Pas de génie. Pas de config de 80 lignes. Chez DanielCraft, un atelier se juge à "ça compile et je comprends", pas à "c'est impressionnant". Lea chronomètre souvent 45 minutes. Max paniquait au premier rouge ; ici tu vas le lire. Sam interdit `any` pendant l'exercice.

Tu vas déclarer des variables typées, une petite fonction, et corriger volontairement des erreurs. Le but n'est pas zéro erreur du premier coup. Le but, c'est le cycle : écrire, `tsc`, lire, corriger.

★ A RETENIR

Annoter, c'est nommer le contrat. Si `tsc` proteste, tu as trouvé un flou : corrige la valeur ou le type.

Mission

1. 1. Crée `atelier-types.ts`.
2. 2. Déclare :
 - `titre: string` - `prix: number` - `promo: boolean`
1. 3. Écris `function label(titre: string, prix: number): string` qui renvoie `"titre - prix EUR"`.
2. 4. Appelle la fonction et `console.log` le résultat.
3. 5. Introduis une mauvaise assignation, lis l'erreur, corrige.

```
let titre: string = "Joint silicone";
let prix: number = 4.5;
let promo: boolean = false;

function label(titre: string, prix: number): string {
  return titre + " - " + prix + " EUR";
}

console.log(label(titre, prix));
```


Ce que ce n'est pas

Ce n'est pas l'heure des interfaces complètes (atelier suivant). Ce n'est pas le DOM. Ce n'est pas "faire joli". Si tu ajoutes dix variables inutiles, tu te disperses. Lea coupe le superflu. Max a tendance à sur-annoter ; Sam lui demande : "cette annotation enseigne-t-elle encore quelque chose ?"

♦ ASTUCE

Quand tu bloques, commente la ligne fautive, recompile, isole. Une erreur à la fois.

Petite histoire

Max a rate `promo: boolean` en mettant `"non"`. L'erreur était claire. Il a sourit malgré lui. Lea a ajouté une annotation de retour manquante puis l'a retirée pour montrer l'inférence. Sam a terminé l'atelier en demandant à chacun de lire une erreur à voix haute. DanielCraft : la voix haute transforme le rouge en phrase.

Erreur classique

Mettre `any` pour finir plus vite. Annoter `string` sur un prix calculé. Oublier d'appeler la fonction. Autre piège : copier le corrigé sans casser puis réparer. L'atelier vit dans la casse volontaire.

⚠ ATTENTION

Ne "répare" pas avec `as any`. Si le type gêne, change la donnée ou le contrat - pas le silence.

En vrai / À toi

Fais la mission. Puis ajoute `remise: number` et une fonction `prixFinal(prix: number, remise: number): number`. Compile. Note une phrase sur ce que `tsc` t'a appris aujourd'hui. Garde le fichier pour le comparer à l'atelier interface.

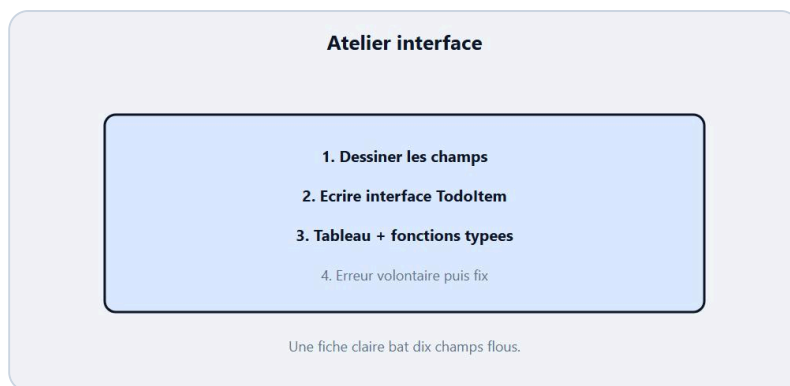
Déroulé détaillé

Installe le rythme. Ouvre ton éditeur. Crée le fichier. Écris les quatre variables annotées avec des valeurs cohérentes. Écris `labelProduit` et appelle-la avec `titre` et `prix`. Écris `appliquerRemise` avec un taux `0.1`. Affiche avant/après. Compile. Si `tsc` est vert, casse : passe `"dix"` en prix. Lis. Corrige.

Ensuite ajoute `estCher`. Teste avec 50 et 150. Ajoute `formatStock`. Enchaîne les `console.log` pour voir un petit ticket produit dans le terminal. Lea demande souvent une capture des logs. Max aime voir les strings se construire. Sam regarde surtout s'il reste un `any` caché.

Si tu débutes vraiment sur `tsc`, reviens trente secondes au chapitre 2. Pas de honte. L'atelier suppose le pipeline, pas la perfection mémoire. Chez DanielCraft, ce muscle nourrit l'atelier interface suivant.

Chapitre 16 - Atelier : une interface pour un vrai objet



Atelier : une interface pour un vrai objet.

Objectif : modeliser un objet metier avec une **interface**, puis ecrire une ou deux fonctions qui la respectent. Chez DanielCraft, l'interface n'est pas un luxe academique. C'est la fiche que tu aurais dessinee sur papier. Lea commence souvent par les champs au tableau. Max code trop vite sans liste ; ici tu listes d'abord. Sam refuse les noms `Data` / `Info`.

Tu vas creer `TodoItem`, une petite liste, et des actions typees. Pas de framework. Juste la forme et les fonctions.

★ A RETENIR

Interface = forme reutilisable. Fonction typee = usage honnete de cette forme.

Mission

1. 1. Cree `atelier-interface.ts`.
2. 2. Ecris :

```
interface TodoItem {
  id: number;
  texte: string;
  fait: boolean;
}

const todos: TodoItem[] = [
  { id: 1, texte: "Lire le chapitre 5", fait: true },
  { id: 2, texte: "Annoter mon script", fait: false },
  { id: 3, texte: "Compiler sans any", fait: false },
];

function restantes(items: TodoItem[]): TodoItem[] {
  return items.filter((t) => t.fait === false);
}

function marquerFait(item: TodoItem): void {
  item.fait = true;
}

console.log(restantes(todos));
```

```
marquerFait(todos[1]);  
console.log(restantes(todos));
```

1. 3. Compile. Ajoute volontairement un champ `titre` au lieu de `texte` sur un littéral, lis l'erreur, corrige.
2. 4. Ajoute `email?: string` seulement si tu as un vrai besoin - sinon laisse.

Ce que ce n'est pas

Ce n'est pas une app todo complete avec DOM (presque : atelier projet). Ce n'est pas le debat `type` vs `interface`. Ce n'est pas vingt champs "au cas ou". Trois champs utiles battent une usine. Lea coupe les proprietes decoratives.

♦ ASTUCE

Si `tsc` parle d'une propriete manquante, regarde l'interface avant le corps de la fonction.

Petite histoire

Lea a fait cet atelier avec un stagiaire qui ecrivait `tache.texte` et `tache.title` selon l'humeur. L'interface a force un seul mot. Max a oublie `fait` ; le compilateur a refuse l'objet. Sam a demande : "ton interface raconte-t-elle la realite ?" DanielCraft : si la fiche ment, le code ment.

Erreur classique

Interface vide. `any` dans un champ "temporaire". Modifier l'interface pour accepter n'importe quoi apres une erreur. Autre piege : muter sans le vouloir dans une fonction qui devrait renvoyer une copie - pour debuter, la mutation simple de `fait` est OK si tu es conscient.

⚠ ATTENTION

Un littéral avec une cle inconnue est souvent refuse. C'est un cadeau : faute de frappe detectee tot.

En vrai / A toi

Ajoute `function compteRestantes(items: TodoItem[]): number`. Teste. Puis cree un quatrieme todo conforme. Si tu as le temps, ajoute `priorite?: "basse" | "haute"` et un filtre. Note ce que l'optionnel change dans tes `if`.

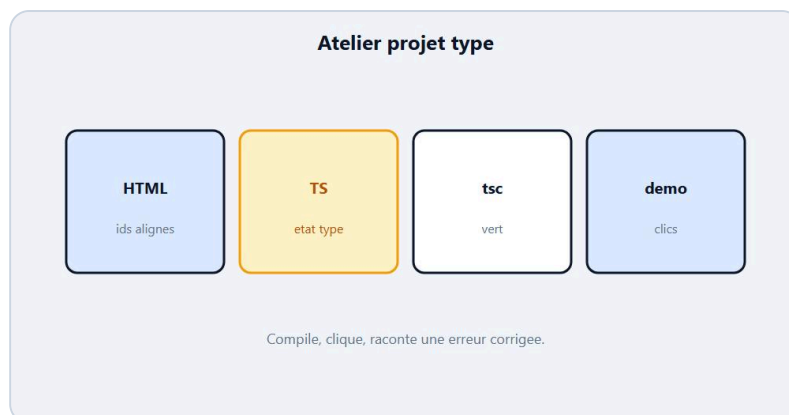
Qualite de modele

Avant de coder, ecris sur papier les champs et pour chacun : obligatoire ou optionnel, type. Puis seulement code l'interface. Lea chronometre cette minute papier : elle evite les interfaces "au feeling". Max saute parfois l'etape et le regrette. Sam note le papier comme partie de la copie.

Quand tu as `priorite?`, teste deux todos : une avec priorite, une sans. Affiche autrement si haute. Tu touches union litterale + optionnel dans un vrai objet. C'est le coeur du chapitre 6 applique. Si `tsc` refuse un littéral `"moyenne"` hors union, c'est gagne : le filet marche.

Termine en renommant un champ dans l'interface et en suivant les erreurs. Tu verras la carte des usages. C'est une raison majeure d'aimer TypeScript en equipe.

Chapitre 17 - Atelier : petit projet type de bout en bout



Atelier : petit projet type de bout en bout.

Objectif : enchainement complet. Une **interface**, un **etat**, des **fonctions**, un peu de **DOM** garde, zero **any**. Chez DanielCraft, cet atelier colle le mini-projet compteur et l'atelier interface : tu livres une mini todo affichee dans la page. Lea demande une V1 en moins de deux heures. Max ajoute trop de features ; Sam coupe : ajouter, afficher, basculer fait.

Tu n'as pas besoin d'un bundler. HTML + **tsc** suffisent. Si le DOM te stresse, repars du chapitre 12. Si l'interface te stresse, repars du 16.

★ A RETENIR

Bout en bout = meme contrat de l'etat jusqu'a l'ecran. Compile souvent.

Mission

HTML minimal : `input#texte`, `button#ajouter`, `ul#liste`. Puis TypeScript :

```
interface TodoItem {
  id: number;
  texte: string;
  fait: boolean;
}

let todos: TodoItem[] = [];
let nextId: number = 1;

const input = document.querySelector<HTMLInputElement>("#texte");
const btn = document.querySelector<HTMLButtonElement>("#ajouter");
const liste = document.querySelector<HTMLUListElement>("#liste");

function rendre(): void {
  if (!liste) return;
  liste.innerHTML = "";
  for (const t of todos) {
    const li = document.createElement("li");
    li.textContent = (t.fait ? "[x] " : "[ ] ") + t.texte;
    li.addEventListener("click", () => {
      t.fait = !t.fait;
      rendre();
    });
  }
}
```

```

    liste.appendChild(li);
  }
}

function ajouter(): void {
  if (!input) return;
  const texte = input.value.trim();
  if (!texte) return;
  todos.push({ id: nextId, texte, fait: false });
  nextId = nextId + 1;
  input.value = "";
  rendre();
}

btn?.addEventListener("click", ajouter);
rendre();

```

Compile, ouvre la page, ajoute deux tâches, clique pour basculer.

Ce que ce n'est pas

Ce n'est pas localStorage obligatoire. Ce n'est pas un design parfait. Ce n'est pas React. Si tu ajoutes dix boutons, tu quittes l'atelier. Lea dit "V1 moche qui compile > V5 jolie qui ment sur les types".

✦ ASTUCE

Si `innerHTML` te gene pour la securite plus tard, OK - ici on reste pedagogique. Le focus, c'est le typage de `todos`.

Petite histoire

Max a livre sans garder `input` contre `null`. `tsc` en strict l'a arrete. Lea a refuse un `todos: any[]`. Sam a demande un `trim()` avant push : les tâches vides polluent. Ensemble, ils ont une demo stable.

DanielCraft applaudit le cycle court, pas le framework.

Erreur classique

Oublier de rappeler `rendre` apres mutation. Typier `texte` en number. Utiliser `as HTMLInputElement` sans query adequat. Autre piege : tout mettre dans un seul listener anonyme de 80 lignes - extraire aide le typage et la lecture.

⚠ ATTENTION

Apres chaque action qui change `todos`, rappelle l'affichage. Sinon ton etat type dit vrai et l'ecran ment.

En vrai / A toi

Ajoute un compteur "N restantes" dans un `span` type. Puis interdis les doublons de texte (optionnel). Ecris trois lignes de bilan : erreur `tsc` vue, correction, prochaine idee. Chez DanielCraft, le bilan ancre plus que la feature bonus.

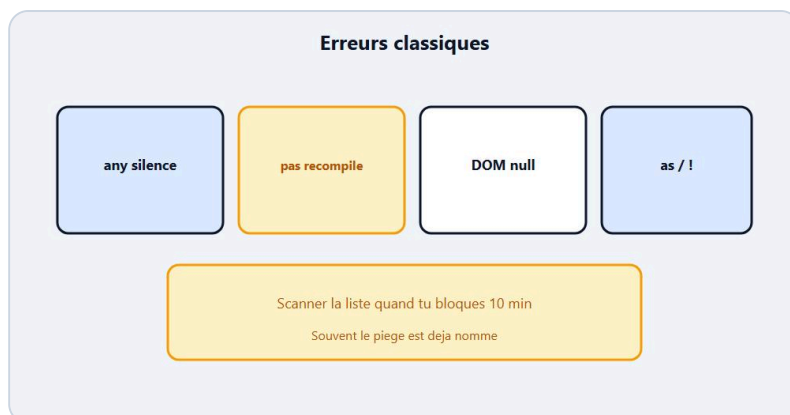
Gerer le temps

Prevois une heure calme. Quarante minutes de construction, dix de polish, dix de debrief. Si tu debloques trop longtemps sur le DOM, reviens au compteur minimal sans CSS. Si tu debloques sur les types, commente une ligne, isole l'erreur, corrige, decomment. Lea enseigne cet isolement. Max a tendance a tout reecrire ; Sam l'arrete.

Prepare aussi la demo : quels clics tu montres, quelle erreur tu as corrige en route (raconte-la). Une demo qui inclut une erreur lue est plus pedagogique qu'une demo parfaite amnesique. Chez DanielCraft, on forme des gens qui racontent leur debug.

Livable minimal accepte : HTML + TS + JS compile + README de cinq lignes (comment lancer `tsc`, comment ouvrir la page). Pas besoin de github. Besoin d'un dossier propre.

Chapitre 18 - Erreurs classiques TypeScript



Pieges : any partout, ignore, mauvais cast.

Tu vas croiser les memes pieges souvent. Ce chapitre les nomme pour que tu les reconnaisse vite. Chez DanielCraft, on prefere une liste courte et vicieuse a un catalogue encyclopedique. Lea les voit en revue de code. Max les a toutes faites. Sam les affiche en debut de seance "erreurs du mois".

Le point commun : vouloir le silence du compilateur au lieu du contrat clair. `any`, ignore, mauvais cast, union fourre-tout, optionnel jamais verifie. Si tu sens l'envie de "juste compiler", pause. Relis le message.

★ A RETENIR

Les classiques : any partout, `@ts-ignore`, `as` sans preuve, unions trop larges, optionnels sans garde.

Ce que ce n'est pas

Ce n'est pas une honte d'avoir fait ces erreurs. Ce n'est pas non plus une interdiction absolue de chaque outil (un `as` rare peut exister). C'est un radar. Lea autorise un escape hatch documente. Max documentait trop tard. Sam exige la raison dans un commentaire d'une ligne.

Les pieges a coller au mur

1. **any de confort.** Ca compile. Les bugs aussi. Remplace par interface ou union.
2. **as Type magique.** Tu forces. Prefere narrowing.
3. **// @ts-ignore / @ts-expect-error sans suite.** Le professeur est baillonne.
4. **string | number | boolean | null | undefined.** Union poubelle. Modele mieux.
5. **Optionnel puis .methode() direct.** Crash. Garde d'abord.
6. **! non-null partout.** Tu affirmes sans preuve.
7. **JSON.parse puis as MonType.** Aucune validation.

```
// fragile
const data = JSON.parse(raw) as TodoItem;

// plus honnête (debut)
const data: unknown = JSON.parse(raw);
// puis narrowing / validation des champs
```

⚠ ATTENTION

Faire taire `tsc` n'est pas corriger. C'est reporter la facture au runtime.

Petite histoire

Un PR de Max ajoutait trois `as any` "temporaires". Lea a demandé les messages originaux. En vingt minutes, deux étaient des narrowing simples, un était une vraie interface manquante. Sam a gardé l'histoire pour la promo suivante. DanielCraft : le temporaire sans date devient permanent.

Erreur classique (meta)

Lire ce chapitre, hocher la tête, puis remettre `any` le soir même sous pression. Antidote : une règle d'équipe "pas de `any` sans TODO date". Autre piège : élargir le type jusqu'à ce que tout passe. Préfère corriger la donnée.

♦ ASTUCE

Quand tu tapes `any` ou `as`, écris une phrase : "je fais quelle erreur exacte ?" Si tu ne sais pas, relis le diagnostic.

En vrai

Ouvre un vieux fichier. Cherche `any`, `as`, `@ts-ignore`. Pour chacun, tente une vraie correction. Chronomètre. Souvent moins long que tu ne le crains.

Comment utiliser cette liste

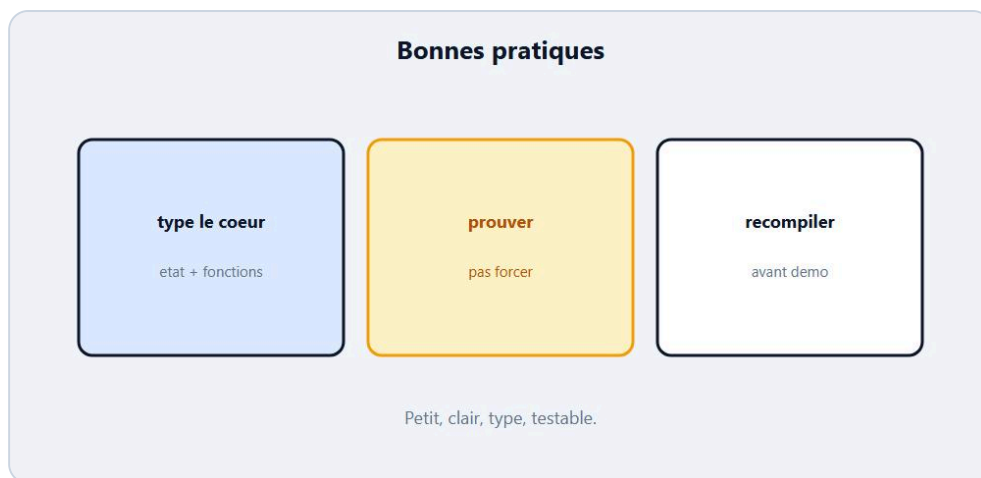
N'essaie pas de tout mémoriser d'un coup. Imprime ou copie les titres. Chaque fois que tu bloques dix minutes, scanne la liste. Souvent le piège est déjà nommé. Lea le fait en mentoring : "c'est le numéro 3, recompile". Max a affiché la liste à côté du moniteur une semaine. Sam en tire des QCM rapides en début de séance.

Ajoute tes propres erreurs quand tu en découvres. Le chapitre devient vivant. TypeScript n'invente pas de nouveaux pièges chaque mois au niveau débutant : ce sont souvent les mêmes dix. Les maîtriser te rend étonnamment rapide.

A toi

Liste tes trois pièges personnels (ceux que tu referais demain). À côté, la correction préférée. Colle la liste près de l'écran. Chez DanielCraft, cette feuille bat un chapitre relu passivement.

Chapitre 19 - Bonnes pratiques debutantes



Pratiques : strict, noms clairs, petits contrats.

Des pratiques simples valent mieux qu'une religion de types. Chez DanielCraft, on vise **strict**, des **noms clairs**, des **petits contrats**, des erreurs lues, peu de magie. Lea active **strict** tot. Max ajoute des interfaces quand une forme revient deux fois. Sam refuse les fichiers de 800 lignes "tout typer plus tard".

Tu n'as pas besoin d'être parfait. Tu as besoin d'être cohérent. Une annotation honnête, une interface nommée, un **if** de narrowing, un **tsc** fréquent : ce quartet porte déjà loin.

★ A RETENIR

Strict, noms clairs, petits contrats, compile souvent. Évite **any** et les casts de panique.

Ce que ce n'est pas

Ce n'est pas "zero **any** a vie sous peine de mort". Ce n'est pas configurer vingt plugins. Ce n'est pas typer pour typer (annoter `const x: number = 1` partout sans gain). Ce n'est pas non plus attendre le setup parfait avant d'écrire. Lea livre avec une config courte comprise.

Gestes qui marchent

1. Active `"strict": true` dans `tsconfig` dès que tu peux.
2. Type les frontières : entrées de fonctions, données externes, props d'objets partagés.
3. Nomme les interfaces comme le métier : `TodoItem`, `DevisLigne`.
4. Préfère union de littéraux aux strings libres pour les états.
5. Narrow au lieu de caster.
6. Corrige la première erreur `tsc` d'abord.
7. Laisse inférer quand c'est évident ; explicite quand ça documente.

```
// frontière claire
function creerTodo(texte: string): TodoItem {
  return { id: Date.now(), texte, fait: false };
}
```

♦ ASTUCE

Si une forme d'objet apparait deux fois, extrais une interface. Une seule fois, un literal annote peut suffire.

Petite histoire

Lea a herite d'un projet sans strict. Chaque semaine un bug "possibly undefined". Elle a active strict module par module. Douloureux deux jours, puis calme. Max a renomme `IData` en `Client`. Sam a coupe un fichier monstre en trois avec des contrats nets. DanielCraft mesure la pratique a la facilite de relire, pas au nombre de types exotiques.

Erreur classique

Sur-typer du bruit. Sous-typer les frontieres importantes. Copier une config incomprise. Autre piege : bonnes pratiques en reunion, `any` en prod le vendredi. Ecris une checklist courte et utilise-la.

⚠ ATTENTION

Une bonne pratique non appliquee ne protege personne. Choisis trois gestes et tiens-les deux semaines.

En vrai

Ouvre ton `tsconfig`. Verifie `strict`. Parcours un fichier : renomme une interface floue, retire un `any`, ajoute un narrowing. Commit mental : "trois gestes".

Ritualiser

Une bonne pratique tient si elle devient rituel. Avant demo : `tsc` vert. Avant commit mental : pas de `any` nouveau. Avant d'ajouter une option `tsconfig` : une phrase "pourquoi". Lea a ces trois rituels. Max en a ajoute un : montrer le projet a quelqu'un meme trente secondes. Sam ajoute : expliquer une erreur en francais.

Tu peux aussi tenir un fichier `NOTES.md` avec tes interfaces cles et tes pieges. Ce n'est pas de la paperasse : c'est une memoire externe. Chez DanielCraft, on prefere une note courte a une documentation fantome de cinquante pages.

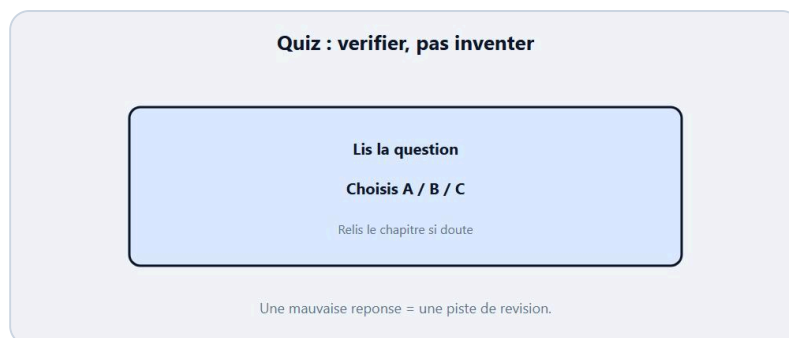
Quand une pratique te ralentit sans securite gagnee, ajuste. Le but n'est pas d'obeir a une liste. Le but est de livrer du code plus sur, plus lisible, plus corrigible.

A toi

Ecris ta checklist perso (5 lignes max). Exemple : strict / pas d'any / interfaces nommees / lire tsc / DOM garde. Colle-la. Chez DanielCraft, la checklist courte bat le manifeste.

QUIZ

Quiz final - Teste-toi !



Quiz : verifier le filtre types.

Meme regle que dans tout le parcours : cherche d'abord, corrige ensuite. Ce quiz n'est pas la pour te coller une note sur le front. C'est pour verifier que **TypeScript** n'est plus un brouillard : types, annotations, interfaces, unions, fonctions, narrowing, `any` vs `unknown`, erreurs `tsc`. Chez DanielCraft, un quiz sert a cibler les chapitres a relire - pas a mesurer ta valeur humaine. Lea revoit parfois ces bases avant une demo. Max a refait le quiz un samedi. Sam l'utilise en fin de sequence.

Lis chaque question calmement. Reponds dans ta tete ou sur papier. Puis descends aux corriges. Note tes hesitations : elles valent plus que les reponses faciles. Si tu bloques sur Q9 et Q10, tu sais ou aller (narrowing / any). Si tout passe, tu peux attaquer le bravo en confiance.

Questions

Avant de commencer, respire. Douze questions, douze checkpoints. Pas de piege volontaire. Juste le socle construit chapitre apres chapitre.

Q1. TypeScript, c'est surtout :

- A) Un framework UI
- B) JavaScript + types verifiees avant l'execution
- C) Un remplacement de HTML

Q2. Que produit generalement `tsc` a partir d'un `.ts` ?

- A) Du Python
- B) Du CSS
- C) Du JavaScript (`.js`)

Q3. Quelle annotation declare un nombre ?

- A) `let n: number = 3`
- B) `let n: string = 3`
- C) `let n: boolean = 3`

Q4. Une **interface** sert surtout a :

- A) Styliser la page
- B) Decrire la forme d'un objet
- C) Remplacer npm

Q5. `email?: string` signifie :

- A) email obligatoire number
- B) email facultatif de type string
- C) email toujours `any`

Les cinq premieres couvrent le socle : idee TS, compilation, annotation, interface, optionnel. Si tu hesites ici, revois les chapitres 1 a 6.

Q6. Dans `function f(x: number): string`, le `: string` annonce :

- A) Le type du parametre
- B) Le type de retour
- C) Un fichier CSS

Q7. `number[]` designe :

- A) Un seul number
- B) Un tableau de numbers
- C) Une union number | string

Q8. Le narrowing avec `typeof x === "string"` sert a :

- A) Effacer le fichier
- B) Prouver que x est string dans la branche
- C) Activer `any`

Q9. Entre `any` et `unknown`, lequel force souvent une verification avant usage ?

- A) `any`
- B) `unknown`
- C) ni l'un ni l'autre

Q10. Face a `Type 'string' is not assignable to type 'number'`, tu devrais surtout :

- A) Ajouter `as any` tout de suite
- B) Lire types fourni vs attendu, puis corriger
- C) Supprimer TypeScript

La moitie du quiz. Fonctions, tableaux, narrowing, any, erreurs. C'est la que Lea revoit le plus avant une livraison.

Q11. Apres `document.querySelector("#x")`, un reflexe sain est :

- A) Ignorer le `null` possible
- B) Garder / tester avant d'utiliser
- C) Cast `as any` systematique

Q12. Pour un etat metier fini (brouillon / envoye / paye), on prefere souvent :

- A) `string` ouverte
- B) Une union de litteraux

- C) `any`

Les deux dernieres questions verifient DOM garde et unions de litteraux. Si tu hesitais, relis chapitres 6 et 12.

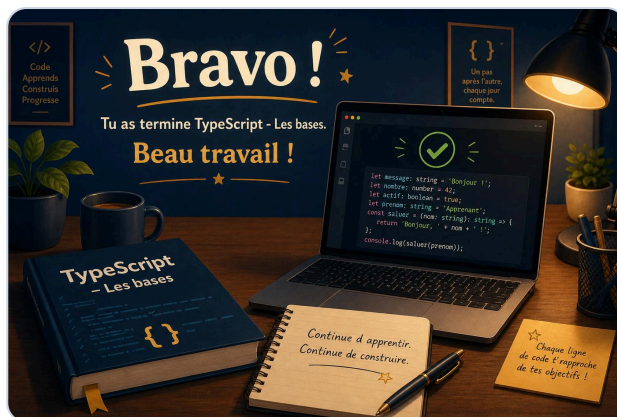
Reponses

1. **B** - TS = JS + types verifies avant l'execution.
2. **C** - `tsc` genere du JavaScript.
3. **A** - `: number` pour un nombre.
4. **B** - L'interface decrit la forme d'un objet.
5. **B** - `?` = facultatif, ici string.
6. **B** - Apres les parentheses, c'est le retour.
7. **B** - `number[]` = tableau de numbers.
8. **B** - Narrowing = preuve dans la branche.
9. **B** - `unknown` demande une verification ; `any` desactive.
10. **B** - Lire le diagnostic, corriger le contrat / la valeur.
11. **B** - Toujours considerer `null` / absence.
12. **B** - Union de litteraux > string libre pour des etats finis.

Score indicatif

- **0 a 5** : reprends les chapitres 1 a 7 sans te juger. Refais Q1-Q6 demain.
- **6 a 9** : socle OK, cible narrowing, `any` / `unknown`, erreurs (8-11) et DOM.
- **10 a 12** : tu as le filtre. Passe au bravo et code un petit projet perso type.

Chez DanielCraft, une mauvaise reponse est une carte de revision, pas un verdict. Lea note ses trous. Max refait seulement les questions ratees. Sam dit : "le quiz mesure un instant, pas ta valeur".



Bravo. Tu ajoutes des types sans magie.

Tu viens de terminer les bases de **TypeScript**. Pas "je maîtrise tous les generics du monde". Mais "je sais ajouter des types a mon JavaScript, compiler, lire une erreur, modeliser un objet, éviter **any**". C'est un vrai cran. Chez DanielCraft, on celebre ce passage parce qu'il est rare : beaucoup installent TS, peu finissent un compteur type et un quiz compris. Toi, tu as tenu le fil de l'annotation jusqu'au mini-projet. Serieux : bravo.

Lea se souvient de sa premiere interface **Devis** qui a bloque un bug avant la demo. Max a montre son score type a sa famille ; le plafond a 20 a fait rire les neveux. Sam applaudit ceux qui ont lu `Type 'string' is not assignable...` sans fermer l'ordi. Tu es dans cette ligue. Le filet commence a t'obeir. Doucement. Proprement.

★ A RETENIR

Tu sais typer le coeur de ton JS et écouter `tsc`. Garde le geste : petit, clair, testable.

Ce que ce n'est pas

Ce n'est pas la fin du chemin TypeScript. Ce n'est pas un framework maîtrise. Ce n'est pas l'obligation d'enchaîner les utilitaires avancées ce soir. Si tu es fatigué, repose-toi. Demain, tu pourras monter. Aujourd'hui, tu souffles et tu regardes ce que tu as construit.

Ce que tu sais faire maintenant

Expliquer TS vs JS. Lancer une intuition `tsc` / `tsconfig`. Annoter variables et fonctions. Ecrire une interface. Utiliser unions et optionnels. Typier des tableaux. Faire du narrowing simple. Preferer `unknown` a `any` quand c'est flou. Lire une erreur compilateur. Toucher le DOM avec null en tête. Livrer un mini compteur ou une todo typée. Ce ne sont plus des slides : ce sont des gestes.

Petite mission

Reprends ton compteur. Ajoute un message a 10 ou une todo minimale avec interface. Montre-le a quelqu'un trente secondes. Lea envoie parfois ses premiers `.ts` aux stagiaires : "regarde d'ou je viens". Max garde une capture du premier `tsc` tout vert. Sam dit que montrer solidifie le courage face aux prochaines erreurs - il y en aura. Heureusement.

Petite histoire

Max a imprimé (oui) le message d'erreur qu'il comprenait enfin. Sam dit que le bravo n'est pas une formalité : il ancre le droit de continuer. Lea rappelle : le premier fichier `.ts` un peu moche qui compile bat le dixième article "TypeScript advanced" jamais ouvert. DanielCraft forme des gens qui livrent petit, souvent, proprement.

La suite quand tu seras prêt

Generics doucement. Types utilitaires quand un vrai besoin apparaît. Meilleure intégration outil de build. Peut-être TS dans un petit projet front plus tard. Mais là, respire. Tu as le socle - comme après le livre JavaScript bases. Le reste s'empile dessus.

En vrai

Ouvre ton projet. Lance `tsc`. Clique. Vérifie. Puis respire. Tu as tenu annotations, interfaces, unions, narrowing. Chez DanielCraft, on célèbre ce passage.

A toi

Ecris trois lignes : (1) une fierté concrète ("mon compteur compile et clique"), (2) un chapitre à revoir si le quiz l'a montré, (3) une date pour vingt minutes de TS. Encore bravo. Garde le rythme. Montre ton fichier à quelqu'un si tu peux.

Tu as les briques. Maintenant, construis.